



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:

Программно-алгоритмический комплекс управления доступом ко
внешним запоминающим устройствам

Студент	ИУ7-85Б		Смирнов П. И.
	(группа)	(подпись)	(инициалы, фамилия)
Руководитель ВКР		(подпись)	Смолина С. Г.
			(инициалы, фамилия)
Нормоконтролер		(подпись)	
			(инициалы, фамилия)

2026 год

РЕФЕРАТ

Расчётно-пояснительная записка содержит 64 страницы, 23 иллюстрации, 7 таблиц, 42 источника, 2 приложения.

Ключевые слова: USB, контроль доступа, доверенные устройства, C++, встраиваемые СУБД, монтирование.

Целью данной дипломной работы является разработка и реализация программно-алгоритмического комплекса для предотвращения несанкционированного копирования информации посредством управления доступом к USB-носителям. Результатом работы является клиент-серверное приложение, где серверная часть выполнена на языке C++, а клиентская на языке TypeScript и фреймворке Vue. Приложение использует встраиваемую СУБД SQLite. Также было проведено исследование алгоритма актуализации записей о монтировании при различных сценариях.

СОДЕРЖАНИЕ

РЕФЕРАТ	5
ВВЕДЕНИЕ	9
1 Аналитический раздел	11
1.1 Сравнительный анализ существующих решений	11
1.2 Диаграмма прецедентов разрабатываемого комплекса	12
1.3 Монтирование устройств в Linux	13
1.3.1 Иерархия каталогов	13
1.3.2 Понятие монтирования	13
1.3.3 Специальные файлы	13
1.3.4 Функции управления монтированием	14
1.3.4.1 Функция mount	14
1.3.4.2 Функция umount	15
1.4 Формализация и описание информации, подлежащей хранению разрабатываемым комплексом	15
1.5 Анализ подходов к организации хранилища информации	16
1.6 Анализ существующих баз данных на основе формализации данных	17
1.6.1 Дореляционные базы данных	18
1.6.2 Реляционные базы данных	18
1.6.3 Постреляционные базы данных	19
1.6.4 Вывод	19
1.7 Анализ существующих типов систем управления базами данных (СУБД) по способу доступа	19
1.7.1 Файл-серверные СУБД	20
1.7.2 Клиент-серверные СУБД	20
1.7.3 Встраиваемые СУБД	21
1.7.4 Сравнительный анализ рассмотренных СУБД	21
1.8 Постановка задачи	22
1.9 Выводы	23

2	Конструкторский раздел	24
2.1	Архитектура разрабатываемого программно-алгоритмического комплекса	24
2.2	Взаимодействие компонентов программно-алгоритмического комплекса	25
2.3	Ключевые структуры данных	25
2.4	Схема алгоритма монтирования устройства	26
2.5	Схема алгоритма удаления устройства из списка доверенных устройств	28
2.6	Схема алгоритма обработки специальных файлов устройств и записей о точках монтирования при инициализации приложения	29
2.7	Схема алгоритма проверки записи о монтировании на предмет актуальности	30
2.8	Описание сущностей проектируемой базы данных	32
2.9	Диаграмма проектируемой базы данных	33
2.10	Описание проектируемых ограничений целостности базы данных	34
3	Технологический раздел	35
3.1	Выбор языков программирования и среды разработки	35
3.2	Выбор средств тестирования кода	36
3.3	Обеспечение связи между клиентом и сервером	37
3.4	Выбор программных средств для работы с устройствами и фай- ловыми системами	37
3.5	Выбор встраиваемой системы управления реляционными базами данных	38
3.6	Классы эквивалентности	39
3.6.1	Классы эквивалентности для получения списка подклю- чённых устройств	39
3.6.2	Классы эквивалентности для получения списка доверен- ных устройств	39
3.6.3	Классы эквивалентности для изменения даты истечения доверия к устройству	40
3.6.4	Классы эквивалентности для удаления устройства из списка доверенных устройств	40

3.6.5	Классы эквивалентности для добавления устройства в список доверенных устройств	41
3.6.6	Классы эквивалентности для метода построения имени папки монтирования	41
3.6.7	Классы эквивалентности для обработки события подключения устройства	42
3.6.8	Классы эквивалентности для получения записи о монтировании по специальному файлу устройства	42
3.7	Структура проекта серверной части комплекса	42
3.8	Развёртывание приложения	46
3.9	Взаимодействие пользователя с приложением	48
3.9.1	Экран подключённых устройств	48
3.9.2	Экран доверенных устройств	49
3.10	Результаты тестирования	51
4	Исследовательский раздел	53
4.1	Постановка исследования	53
4.2	Технические характеристики	53
4.3	Проведение исследования и результаты	53
4.3.1	План первой части исследования	54
4.3.2	Результаты первой части исследования	54
4.3.3	План второй части исследования	55
4.3.4	Результаты второй части исследования	56
4.3.5	План третьей части исследования	57
4.3.6	Результаты третьей части исследования	57
	ЗАКЛЮЧЕНИЕ	59
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	64
	ПРИЛОЖЕНИЕ А	65
	ПРИЛОЖЕНИЕ Б	66

ВВЕДЕНИЕ

Наиболее часто встречающимся интерфейсом для подключения внешних запоминающих устройств к вычислительной технике является USB (Universal Serial Bus). Широкое использование USB-устройств для хранения и передачи данных обусловлено их универсальностью и надежностью. В то же время USB-устройства являются одними из наиболее опасных и активно используемых средств и каналов реализации угроз ИБ (информационной безопасности) [1]. О значительном росте (более чем в два раза) инцидентов ИБ в России, связанных с утечками конфиденциальной информации посредством USB-носителей, сообщается в аналитическом отчете отечественной компании InfoWatch [2]. Для предотвращения утечек информации используются два подхода. Первый подход предполагает полный запрет и отключение физической возможности использования USB-устройств. Реализация данного решения целесообразна для систем, хранящих и обрабатывающих информацию, содержащую сведения, составляющие государственную тайну или особо конфиденциальные данные. Второй подход основан на использовании программных и аппаратно-программных средств защиты от несанкционированного доступа (СЗИ НСД).

Цель данной работы — разработка и реализация программно-алгоритмического комплекса для предотвращения несанкционированного копирования информации посредством управления доступом к USB-носителям.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать существующие решения в сфере управления доступом ко внешним запоминающим устройствам;
- формализовать требования к разрабатываемому комплексу с использованием диаграммы прецедентов;
- разработать архитектуру программно-алгоритмического комплекса;
- описать компоненты и их взаимодействие;
- описать ключевые структуры данных;
- обосновать выбор средств программной реализации;

- разработать программное обеспечение комплекса;
- провести тестирования комплекса;
- описать взаимодействие пользователя с программным обеспечением;
- исследовать характеристики разработанного программного обеспечения.

1 Аналитический раздел

В данном разделе проводится анализ существующих решений, формализуются требования к разрабатываемому комплексу. Также приводится описание информации, используемой программным комплексом в процессе работы и анализируются способы её хранения.

1.1 Сравнительный анализ существующих решений

Существующие программные средства защиты от утечки и утери данных условно можно разделить на две основные категории: корпоративные DLP (Data Leak Prevention)-системы и решения, предназначенные для персонального использования. Корпоративные системы ориентированы на эксплуатацию в инфраструктуре организаций и, как правило, имеют распределённую архитектуру, включающую серверные компоненты, средства централизованного администрирования, мониторинга и управления большим количеством рабочих станций. Такие решения обладают широким набором функций контроля и аудита, однако их использование на отдельном персональном компьютере зачастую является избыточным как с функциональной, так и с ресурсной точки зрения. В рамках данного анализа рассматриваются только решения, предназначенные для личного использования.

Для анализа были выбраны следующие программные комплексы:

1. USBGuard [3];
2. USBFilter [4];
3. USBWall [5];
4. USBShield [6];
5. Ratool [7];
6. GiliSoft USB Lock [8];
7. USB Block [9].

В таблице 1.1 представлены результаты анализа существующих решений.

Таблица 1.1 – Результат анализа существующих решений

Номер решения Критерий	1	2	3	4	5	6	7
поддержка ОС (операционной системы) Linux Ubuntu	да	да	да	нет	нет	нет	нет
возможность создания списка доверенных устройств	да	да	да	да	нет	да	да
возможность установки даты истечения периода доверия к устройству	нет	нет	нет	нет	нет	нет	нет
наличие графического интерфейса	нет	нет	нет	да	да	да	да
наличие обновлений и поддержки	да	нет	нет	да	да	да	да

Таким образом, представленные решения не соответствуют в полной мере поставленным критериям.

1.2 Диаграмма прецедентов разрабатываемого комплекса

На рисунке 1.1 представлена диаграмма прецедентов разрабатываемого комплекса.

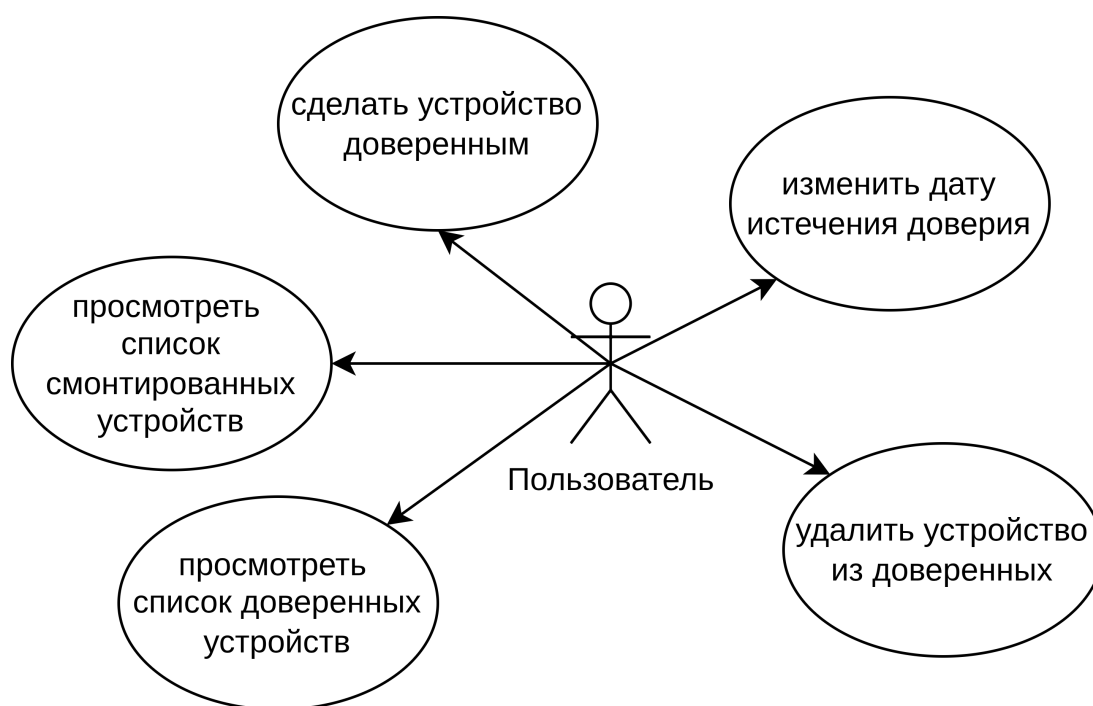


Рисунок 1.1 – Диаграмма прецедентов комплекса

1.3 Монтирование устройств в Linux

Монтирование является связующим звеном между отдельными файловыми системами устройств и единой иерархией каталогов.

1.3.1 Иерархия каталогов

В Linux файлы организованы в виде древовидной структуры (дерева), называемой иерархией каталогов [10]. Каждый файл имеет имя, определяющее его расположение в дереве каталогов. Корнем этого дерева является корневой каталог (root directory), имеющий имя «/». Каждый файл или каталог имеет уникальное имя, определяющее его расположение в дереве, а именно путь — список каталогов, которые необходимо пройти, чтобы достичь нужного объекта.

1.3.2 Понятие монтирования

Монтирование — это механизм подключения файловой системы к единой корневой иерархии каталогов [11]. Монтирование связывает файловую систему с указанным каталогом, называемым точкой монтирования. После этого все файлы подключённой системы становятся доступны через путь, начинающийся от точки монтирования. Если точка монтирования содержит собственные файлы, они становятся недоступны на время подключения, поэтому на практике используются пустые каталоги.

1.3.3 Специальные файлы

Устройства ввода-вывода в Linux представлены специальными файлами [11]. При этом с ними можно проводить операции чтения и записи, используя те же системные вызовы, которые применяются для чтения и записи файлов. Существуют два вида специальных файлов: блочные специальные файлы и символьные специальные файлы. Блочные специальные файлы используются для моделирования устройств, содержащих набор блоков с произвольной адресацией, таких как диски. Аналогичным образом символьные специальные файлы используются для моделирования принтеров, модемов и других устройств, которые принимают или выдают поток символов. В соответствии с принятым соглашением специальные файлы хранятся в каталоге

/dev [12].

1.3.4 Функции управления монтированием

Монтирование и размонтирование файловой системы осуществляется с помощью функций `mount` [13] и `umount` [14] соответственно. Функции объявлены в заголовочном файле `sys/mount.h`.

1.3.4.1 Функция `mount`

Заголовок функции `mount()` представлен в листинге 1.1:

Листинг 1.1 – Заголовок `mount()`

```
int mount(  
    const char *source,  
    const char *target,  
    const char *filesystemtype,  
    unsigned long mountflags,  
    const void *data);
```

где:

- `source` — специальный файл устройства;
- `target` — имя точки монтирования;
- `filesystemtype` — имя файловой системы устройства. Доступные ядру системы перечислены в файле `/proc/filesystems`;
- `mountflags` — флаги монтирования;
- `data` — строка параметров, перечисленных через запятую. Названия и значения параметров индивидуальны для каждой файловой системы.

Флаги монтирования используются для изменения поведения `mount()`. К основным флагам монтирования относятся:

- `MS_MOVE` — изменение точки монтирования. При указании этого флага в `source` содержится не путь к файлу устройства, а существующая точка монтирования;

- `MS_RDONLY` — монтирует файловую систему в режиме «Только для чтения»;
- `MS_NOEXEC` — запрещает выполнение программ в этой файловой системе;
- `MS_REMOUNT` — перемонтирование существующего монтирования. Позволяет изменить `mountflags` и `data` существующих монтирований без необходимости размонтировать и заново монтировать файловую систему.

1.3.4.2 Функция `umount`

Заголовок функции `umount()` представлен в листинге 1.2:

Листинг 1.2 – Заголовок `umount()`

```
int umount(const char *target);
```

где `target` — имя точки монтирования.

Существует также функция `umount2()`, которая, как и `umount()`, размонтирует заданный объект, принимающая дополнительные флаги, контролируемые поведение операции. Заголовок функции `umount2()` представлен в листинге 1.3:

Листинг 1.3 – Заголовок `umount2()`

```
int umount2(const char *target, int flags);
```

Флаг `MNT_FORCE` — немедленное размонтирование, даже если в данный момент производятся операции чтения/записи над файлами данной файловой системы.

Флаг `MNT_DETACH` — точка монтирования становится недоступной для новых операций, однако файловая система остаётся примонтированной для процессов, которые уже используют её файлы.

1.4 Формализация и описание информации, подлежащей хранению разрабатываемым комплексом

Формальное описание устройства может быть представлено следующими атрибутами:

1. идентификатор производителя;
2. идентификатор устройства;

3. уникальный серийный номер устройства;
4. наименование производителя;
5. наименование устройства.

Доверенное устройство характеризуется формальным описанием устройства и датой истечения доверия.

Запись о монтировании устройства описывается следующей информацией:

1. формальное описание устройства;
2. узел устройства;
3. путь к папке монтирования;
4. режим монтирования («Только чтение» или «Чтение и запись»).

1.5 Анализ подходов к организации хранилища информации

Наиболее распространёнными подходами к организации хранилища данных, используемых программным комплексом, являются использование баз данных и хранение информации в плоских файлах.

В таблице 1.2 представлены результаты сравнительного анализа способов организации хранилищ данных.

Таблица 1.2 – Результат анализа подходов к хранению информации

Критерий	Решение	
	Плоские файлы	Базы данных
наличие встроенных механизмов поддержания целостности данных (ограничения, уникальность)	нет	да
устойчивость хранилища к сбоям и некорректному завершению работы	нет	да
отсутствие дополнительных зависимостей у прикладного приложения	да	нет
отсутствие возможности несанкционированного ручного просмотра и редактирования данных без специализированных инструментов	нет	да
возможность выполнения выборок и фильтрации данных без дополнительной реализации алгоритмов поиска	нет	да
отсутствие необходимости инициализации и настройки хранилища перед началом работы	да	нет
возможность расширения структуры данных и добавления новых требований без переработки существующей логики хранения	нет	да

На основе таблицы анализа для разрабатываемого комплекса предпочтительнее использовать базу данных.

1.6 Анализ существующих баз данных на основе формализации данных

Базы данных подразделяются на три основных типа по модели данных:

1. дореляционные;
2. реляционные;
3. постреляционные.

1.6.1 Дореляционные базы данных

Дореляционные базы данных [15] хранят записи в виде связей предок/потомок. Основными недостатками дореляционных баз данных являются избыточность данных и сложность выполнения запросов. Изменение схемы базы данных крайне ограничена. Целостность данных обеспечивается за счёт жёсткой структуры связей. Пример хранения записей в дореляционных базах представлен на рисунке 1.2.

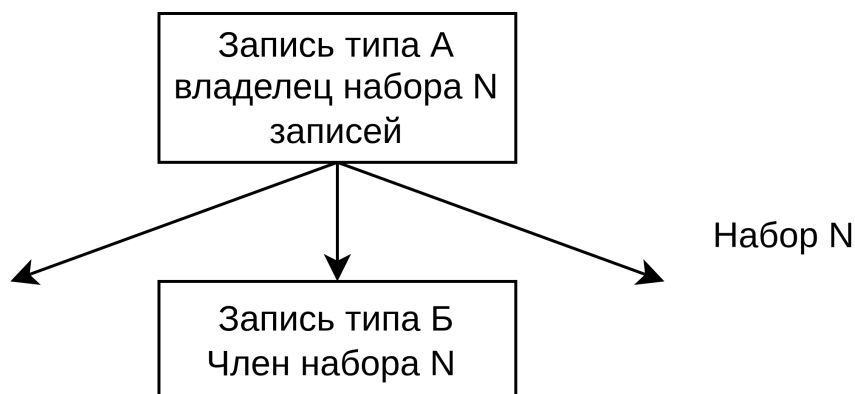


Рисунок 1.2 – Пример хранения записей в дореляционных базах данных

1.6.2 Реляционные базы данных

В реляционных базах данных данные хранятся в виде двумерных таблиц. Связь между ними обеспечивается общими столбцами, такими как идентификаторы. Табличная архитектура подходит для хранения структурированной информации. Целостность и непротиворечивость данных соблюдается при помощи ограничений. Пример хранения записей в реляционных базах представлен на рисунке 1.3.

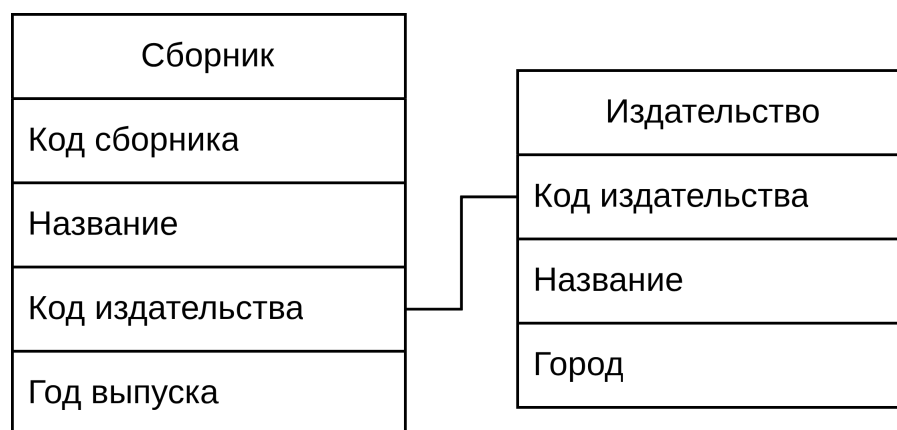


Рисунок 1.3 – Пример хранения записей в реляционных базах данных

1.6.3 Постреляционные базы данных

Постреляционные базы данных подходят для хранения неструктурированных данных, поддерживая логические связи между ними. Недостатком является сложность обеспечения целостности данных. Отличаются хорошей горизонтальной масштабируемостью, сложность запросов варьируется в зависимости от конкретной реализации.

1.6.4 Вывод

Для реализации поставленной задачи была выбрана реляционная модель, так как она лучше всего подходит для хранения структурированной информации, обеспечивая баланс между целостностью данных, простотой запросов и возможностями масштабирования.

1.7 Анализ существующих типов систем управления базами данных (СУБД) по способу доступа

СУБД по способу доступа к базе данных подразделяется на три основных вида [16]:

1. файл-серверные;
2. клиент-серверные;
3. встраиваемые.

1.7.1 Файл-серверные СУБД

При работе с файл-серверной СУБД обработка данных происходит на рабочем месте, а сервер применяется только как отдельный накопитель. Каждый пользователь сам использует информацию и вносит изменения в файлы данных и в индексные файлы. На рисунке 1.4 представлена общая схема работы файл-серверных СУБД.

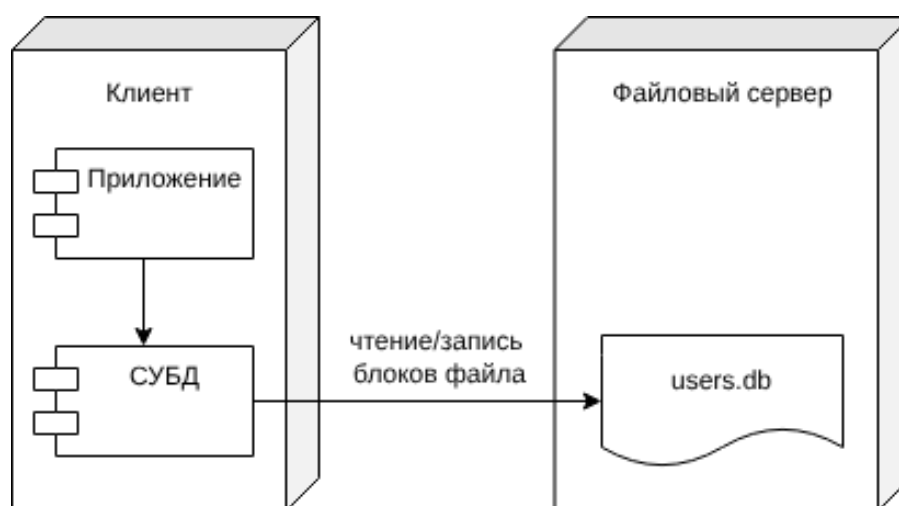


Рисунок 1.4 – Схема работы файл-серверных СУБД

1.7.2 Клиент-серверные СУБД

Клиент-серверная СУБД позволяет обмениваться клиенту и серверу минимально необходимыми объёмами информации. Клиент может выполнять функции предварительной обработки перед передачей информации серверу, но в основном его функции заключаются в организации доступа пользователя к серверу. В большинстве случаев клиент-серверная СУБД гораздо менее требовательна к пропускной способности компьютерной сети, чем файл-серверная СУБД, особенно при выполнении операции поиска в базе данных по заданным пользователем параметрам, т.к. для поиска нет необходимости получать на клиент весь массив данных: клиент передаёт параметры запроса серверу, а сервер производит поиск по полученному запросу в локальной базе данных. Результат выполнения запроса возвращается клиенту, который обеспечивает отображение результата пользователю. На рисунке 1.5 представлена общая схема работы клиент-серверных СУБД.

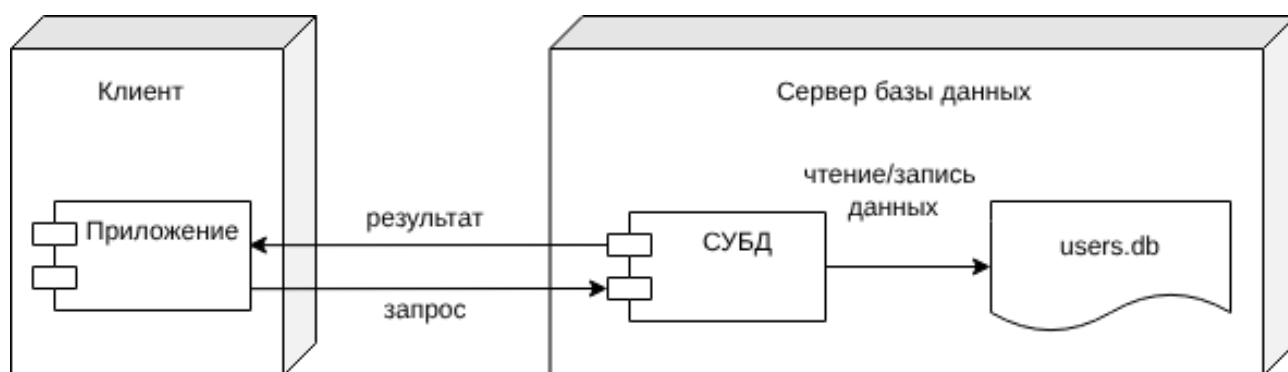


Рисунок 1.5 – Схема работы клиент-серверных СУБД

1.7.3 Встраиваемые СУБД

Встраиваемая система управления базой данных — это система, которая может быть связана с клиентским приложением таким образом, чтобы приложение и СУБД работали в едином адресном пространстве. Вместе со встроенной базой данных приложение может быть развернуто как единая программа. Благодаря связыванию приложения с базой данных, прикладная система выигрывает от снижения общей сложности и уменьшения затрат на администрирование. На рисунке 1.6 представлена общая схема работы встраиваемых СУБД.

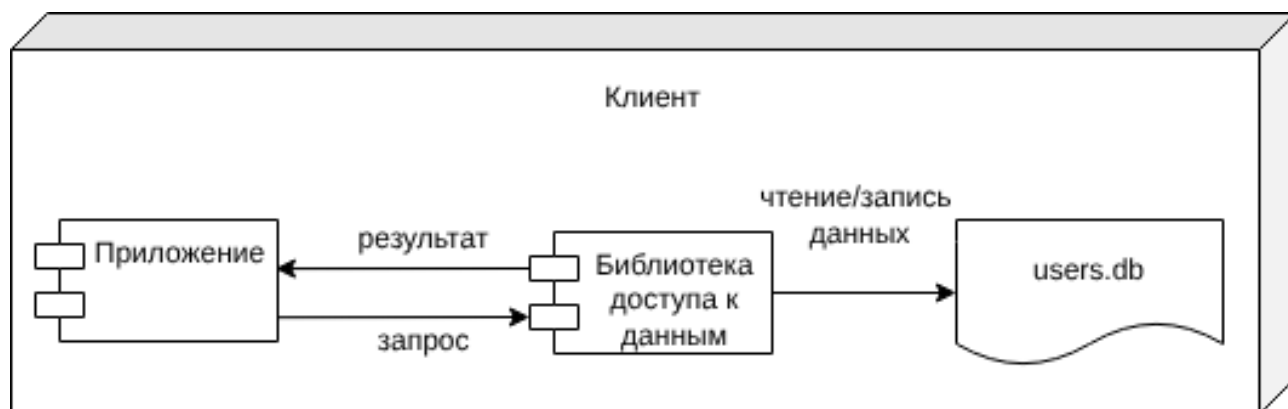


Рисунок 1.6 – Схема работы встраиваемых СУБД

1.7.4 Сравнительный анализ рассмотренных СУБД

В таблице 1.3 представлены результаты сравнения рассмотренных СУБД.

Таблица 1.3 – Результаты сравнения СУБД по способу доступа

Критерий \ Тип СУБД	Файл-серверная	Клиент-серверная	Встраиваемая
Отсутствие избыточности для локального однопользовательского приложения	да	нет	да
Отсутствие необходимости сетевого взаимодействия	нет	нет	да
Отсутствие необходимости в отдельном серверном процессе	да	нет	да

На основании сравнения СУБД для разрабатываемого программно-алгоритмического комплекса наиболее подходящей будет встраиваемая СУБД.

1.8 Постановка задачи

Таким образом, постановка задачи разработки и реализации программно-алгоритмического комплекса, снижающего вероятность утечки информации посредством управления доступом к USB-носителям звучит следующим образом:

Требуется создать программный комплекс, использующий встраиваемую систему управления реляционными базами данных, предоставляющий пользователю следующие возможности:

1. просмотр списка доверенных устройств;
2. добавление устройства в доверенные;
3. удаление устройства из доверенных;
4. установка даты истечения доверия;
5. просмотр смонтированных устройств.

1.9 Выводы

В данном разделе было проведено сравнение существующих программных решений для защиты от утечек данных по сформулированным критериям. Функциональность программно-алгоритмического комплекса была формализована с использованием диаграммы прецедентов. Были рассмотрены основные понятия и функции, связанные с монтированием файловых систем в Linux. Формализована и описана информация, подлежащая хранению. Проанализированы способы организации хранилища, выбраны наиболее подходящие типы базы данных и СУБД.

2 Конструкторский раздел

В данном разделе представлена архитектура программно-алгоритмического комплекса, описаны его компоненты, их взаимодействие и ключевые структуры данных, приведены схемы спроектированных алгоритмов, описаны сущности и ограничения целостности проектируемой базы данных.

2.1 Архитектура разрабатываемого программно-алгоритмического комплекса

Архитектура комплекса будет представлена моделью клиент-сервер, где клиент — приложение, позволяющее пользователю управлять списком доверенных устройств, а сервер — приложение, отвечающее непосредственно за монтирование устройств и работе с базой данных. Схематично данная модель представлена на рисунке 2.1.

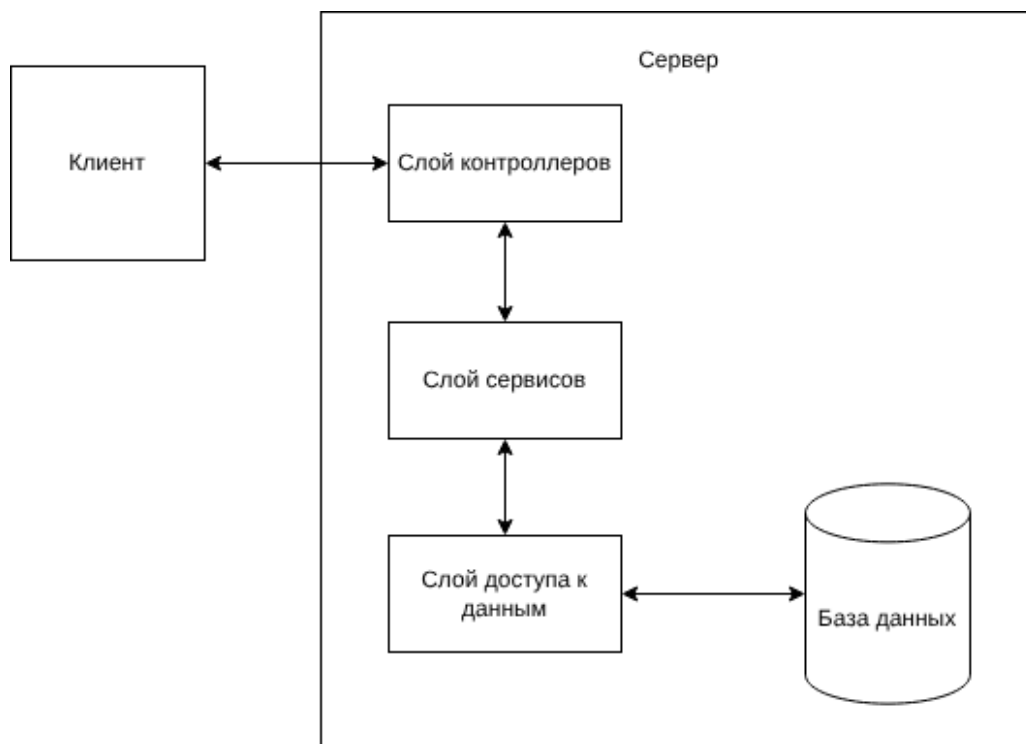


Рисунок 2.1 – Модель клиент-сервер

2.2 Взаимодействие компонентов программно-алгоритмического комплекса

Взаимодействие клиента и сервера описывается с помощью спецификации OpenAPI [17]. Использование OpenAPI позволяет формализовать структуру запросов и ответов, определить параметры запросов и форматы передаваемых данных.

На рисунке 2.2 представлена спецификация OpenAPI, описывающая маршруты взаимодействия между клиентской частью и сервером.

Список доверенных устройств			^
GET	/api/v1/whitelist/	Получение списка	▼
POST	/api/v1/whitelist/	Добавление устройства в список	▼
DELETE	/api/v1/whitelist/{id}	Удаление устройства из списка	▼
PATCH	/api/v1/whitelist/{id}	Обновление даты истечения доверия к устройству в списке	▼
Список подключённых устройств			^
GET	/api/v1/list/	Получение списка подключённых в данный момент устройств	▼

Рисунок 2.2 – Маршруты взаимодействия

2.3 Ключевые структуры данных

Серверная часть использует несколько структур данных для отправки ответов клиенту и использования во внутренних методах.

— DeviceInfo — структура формального описания USB-устройства:

1. `vendorId` — идентификатор производителя;
2. `productId` — идентификатор устройства;
3. `serial` — серийный номер;
4. `vendorName` — название производителя;
5. `productName` — название продукта.

- **Device** — структура описания доверенного устройства, используется для передачи на клиент при запросе списка доверенных устройств:
 1. **id** — первичный ключ из базы данных;
 2. **info** — формальное описание **DeviceInfo**;
 3. **validTo** — дата истечения доверия.
- **EventType** — перечисляемый тип, используемый для обозначения физического манипулирования с накопителем, где **INSERT(0)** — подключение устройства, **REMOVE(1)** — извлечение устройства;
- **DeviceEvent** — структура, описывающая физическое событие с накопителем. Имеет поля **type** типа **EventType**, и **devNode** — специальный файл устройства;
- **MODE** — перечисляемый тип, используемый для обозначения режима монтирования, где **RO(0)** — режим только для чтения, а **RW(1)** — режим полного доступа;
- **MountRecord** — структура описания события монтирования, используется для передачи на клиент при запросе подключённых в данный момент устройств:
 1. **id** — первичный ключ устройства из базы данных(при наличии);
 2. **devNode** — специальный файл устройства;
 3. **mountPoint** — путь монтирования;
 4. **info** — формальное описание **DeviceInfo**;
 5. **mode** — режим монтирования **MODE**.

2.4 Схема алгоритма монтирования устройства

На рисунке 2.3 представлена схема алгоритма монтирования устройства при его подключении к компьютеру.

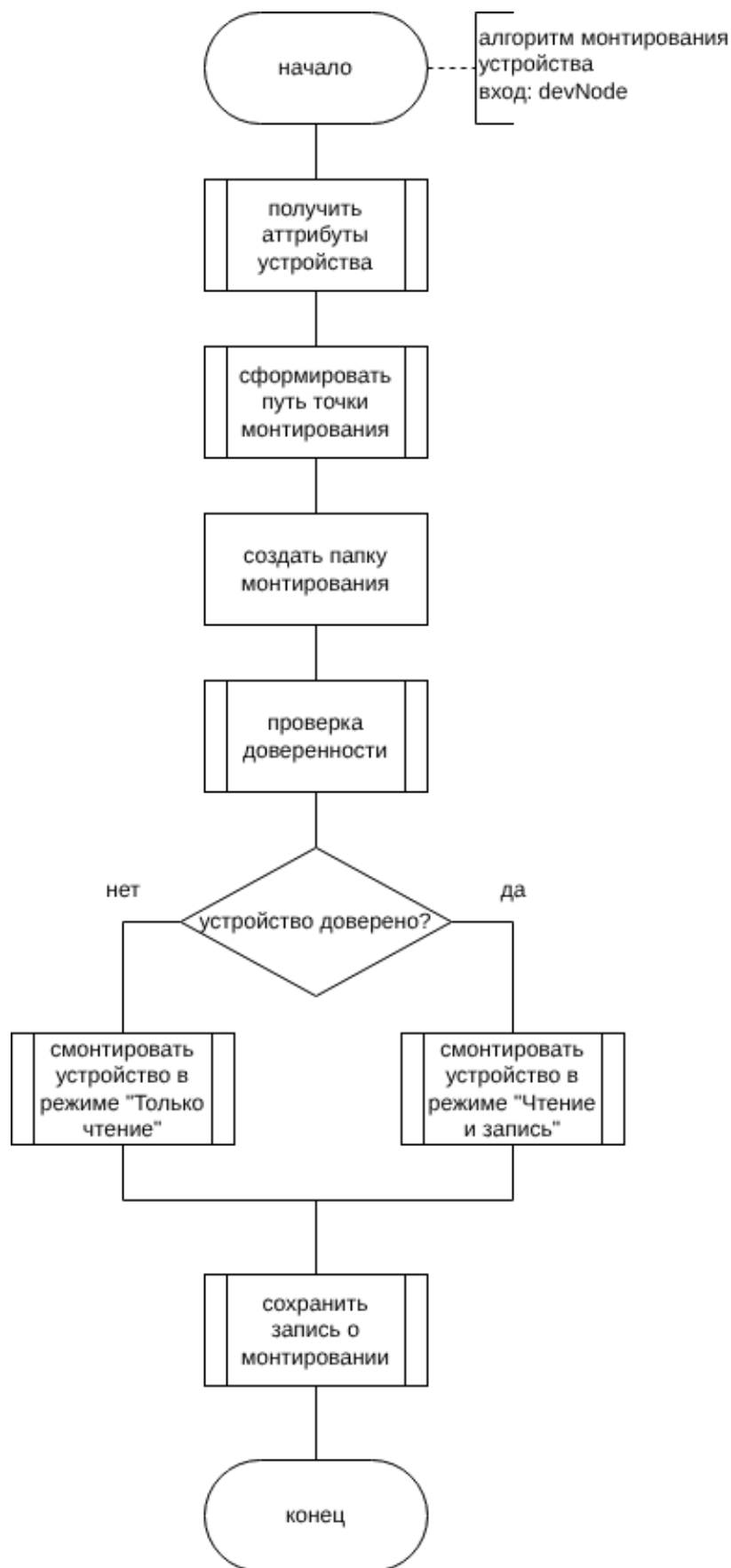


Рисунок 2.3 – Схема алгоритма монтирования

2.5 Схема алгоритма удаления устройства из списка доверенных устройств

На рисунке 2.4 представлена схема алгоритма удаления устройства из списка доверенных устройств.

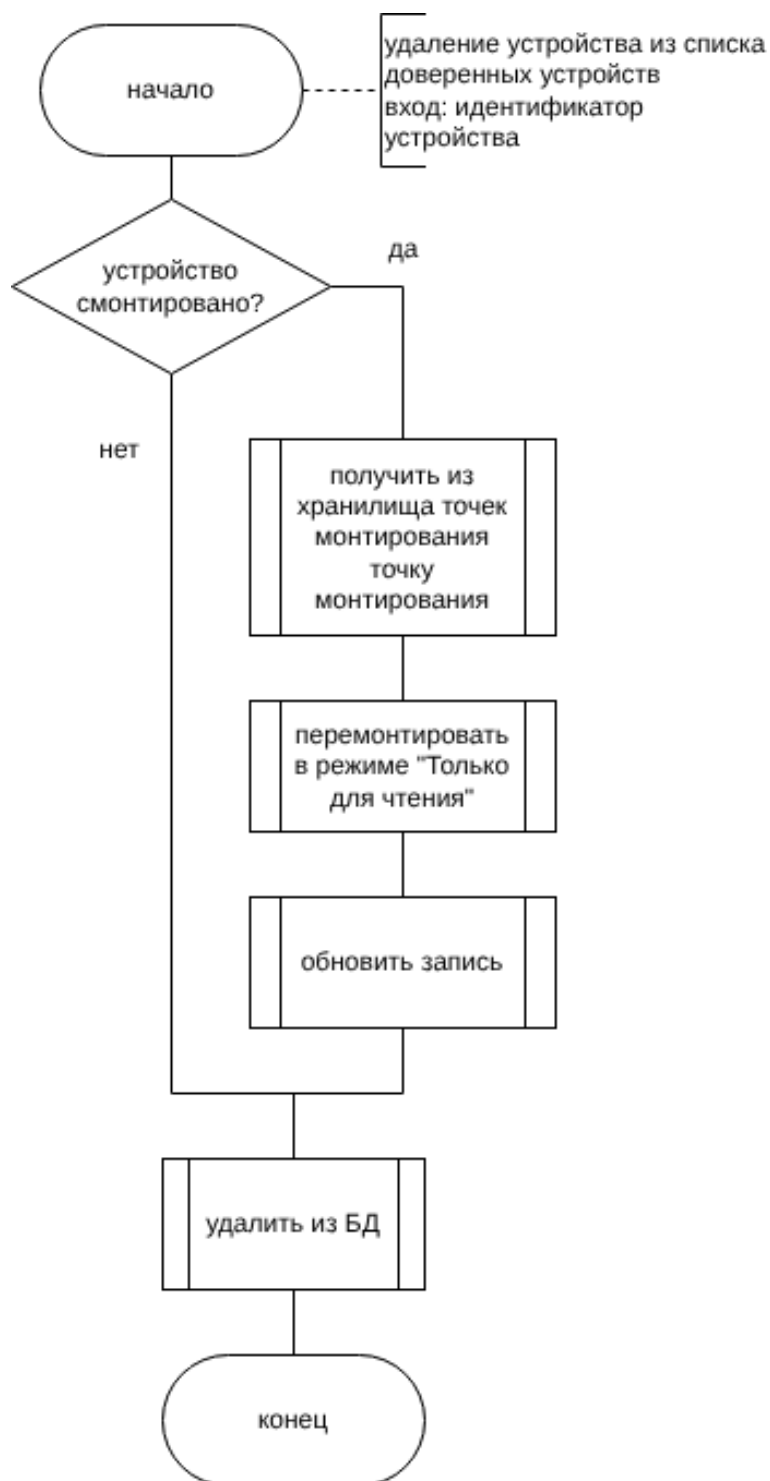


Рисунок 2.4 – Схема алгоритма удаления устройства

2.6 Схема алгоритма обработки специальных файлов устройств и записей о точках монтирования при инициализации приложения

На рисунке 2.5 представлена схема алгоритма обработки специальных файлов устройств и записей о точках монтирования при инициализации приложения.

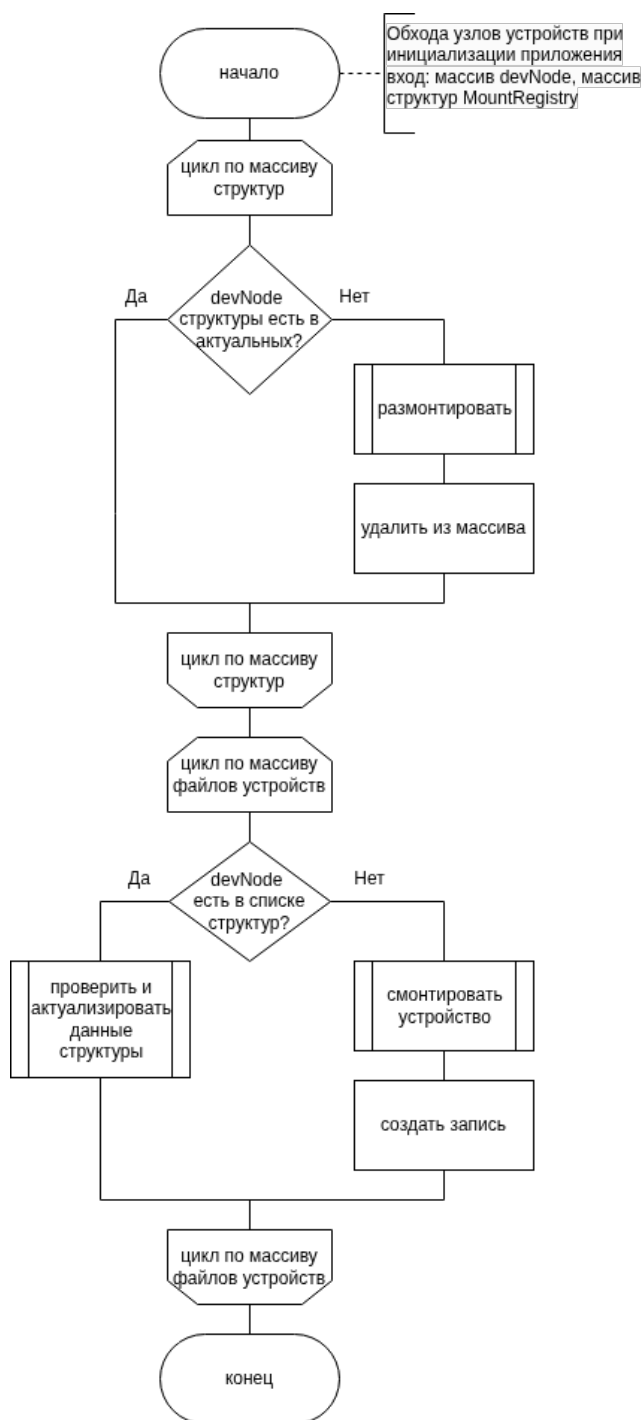


Рисунок 2.5 – Схема алгоритма обработки специальных файлов устройств и записей о точках монтирования при инициализации приложения

2.7 Схема алгоритма проверки записи о монтировании на предмет актуальности

На рисунках 2.6-2.8 представлена схема алгоритма проверки записи о монтировании на предмет актуальности.

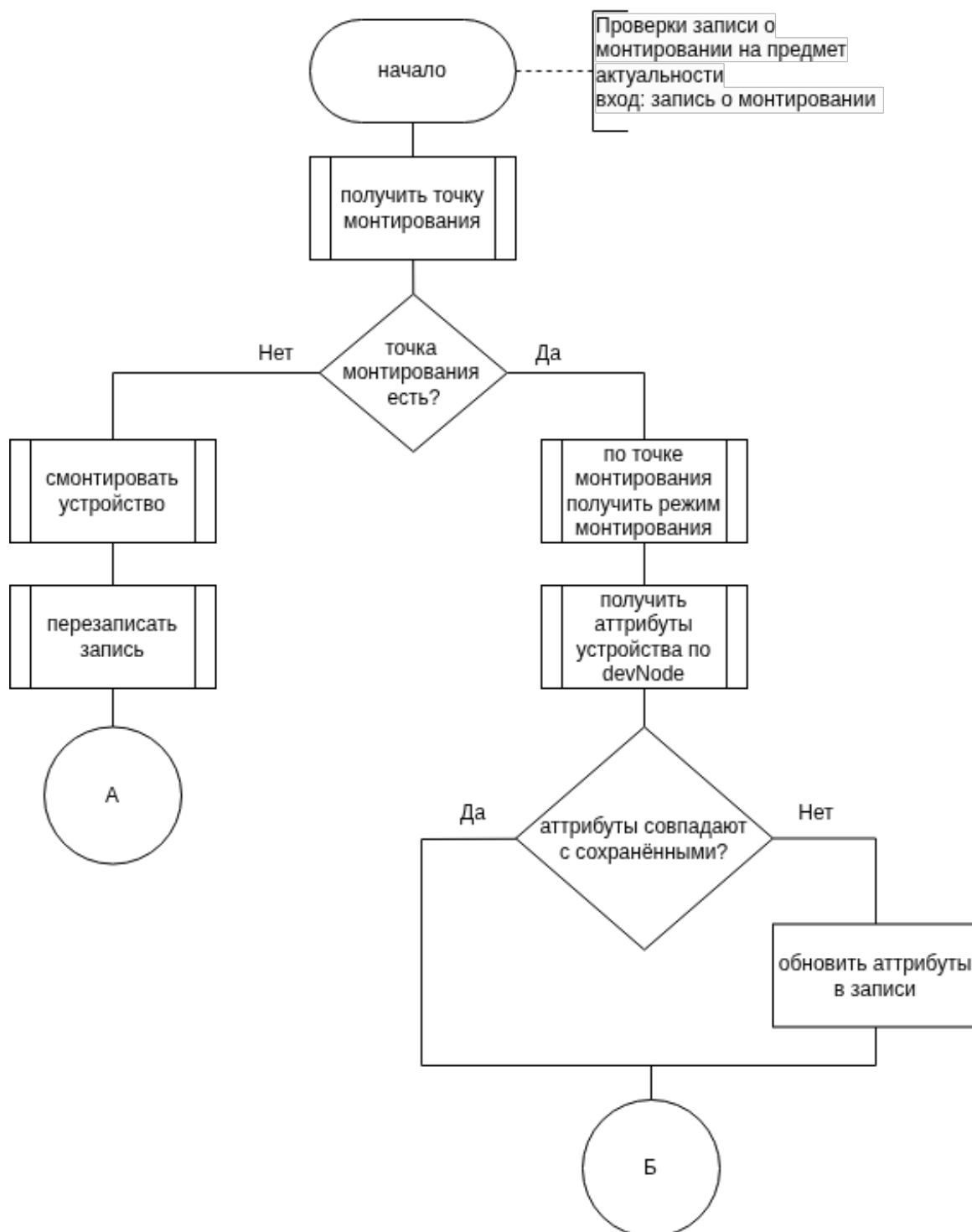


Рисунок 2.6 – Схема алгоритма проверки записи о монтировании на предмет актуальности

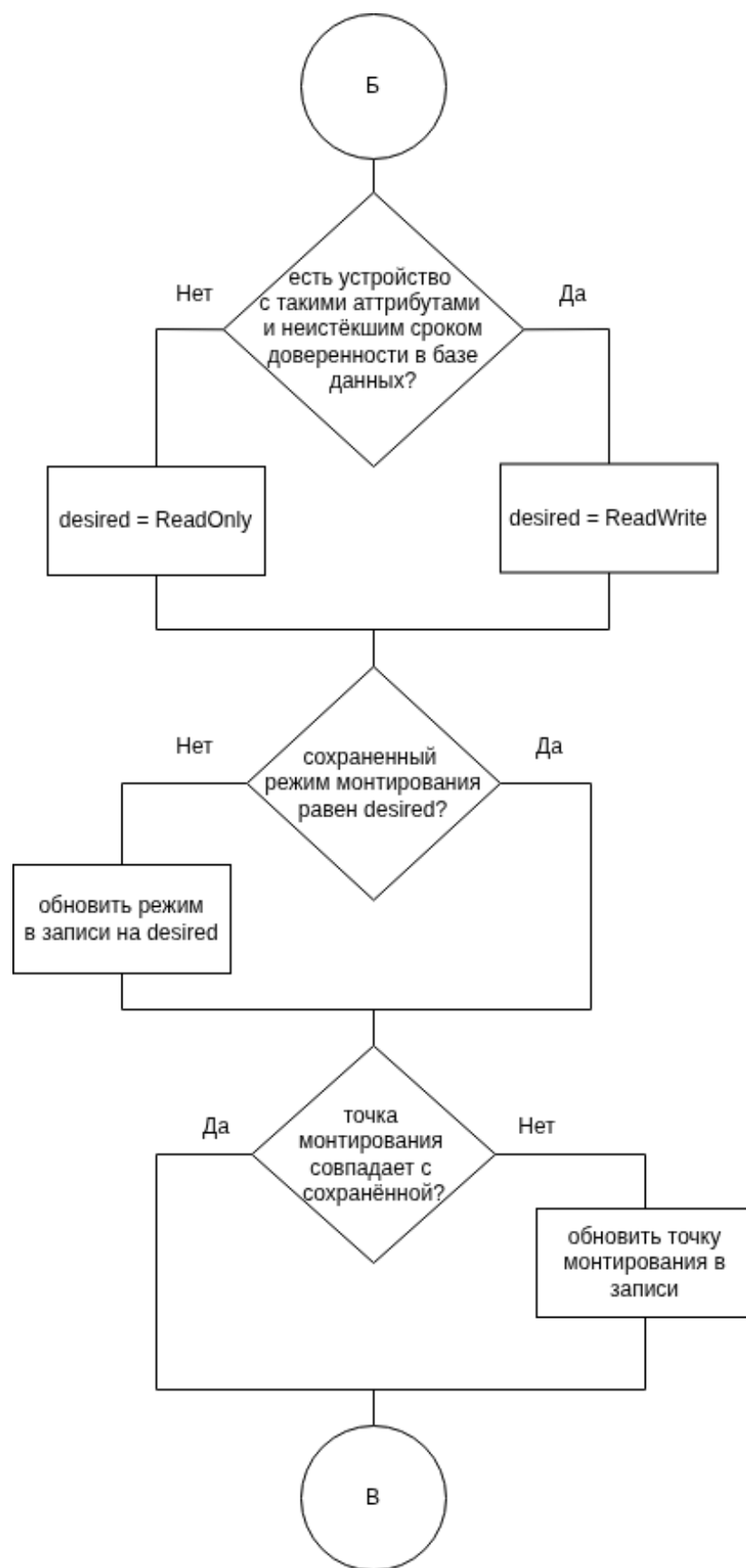


Рисунок 2.7 – Продолжение схемы алгоритма 2.6

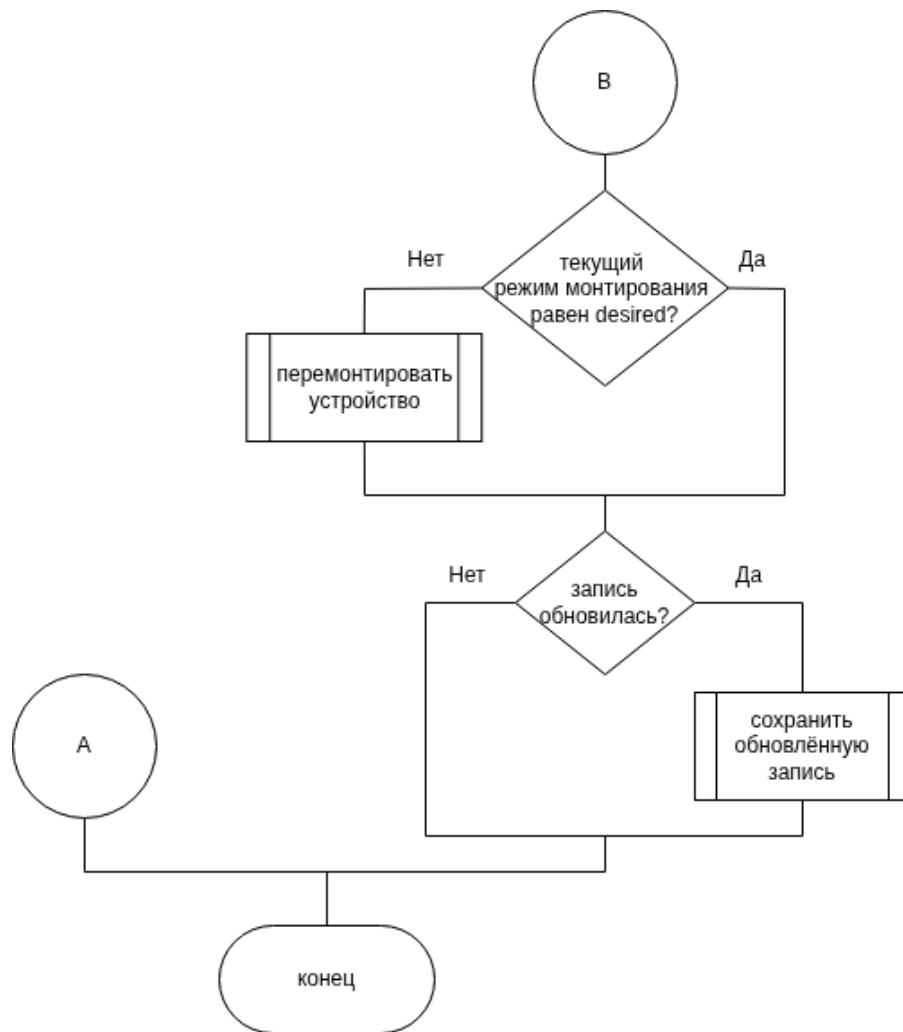


Рисунок 2.8 – Продолжение схемы алгоритма 2.7

2.8 Описание сущностей проектируемой базы данных

База данных содержит следующие таблицы:

— DeviceInfo — описание устройства:

1. `id` — первичный ключ;
2. `vendorId` — идентификатор производителя;
3. `productId` — идентификатор продукта;
4. `serial` — серийный номер;
5. `productName` — название производителя;
6. `vendorName` — название устройства.

- **Device** — описание записи в списке доверенных устройств:
 1. **id** — первичный ключ;
 2. **deviceInfoId** — идентификатор описания устройства;
 3. **validTo** — дата окончания периода доверия.
- **MountRecord** — описание события монтирования:
 1. **id** — первичный ключ;
 2. **deviceInfoId** — идентификатор описания устройства;
 3. **devNode** — специальный файл устройства;
 4. **mountPoint** — точка монтирования;
 5. **mode** — режим монтирования.

2.9 Диаграмма проектируемой базы данных

На рисунке 2.9 представлена диаграмма проектируемой базы данных.

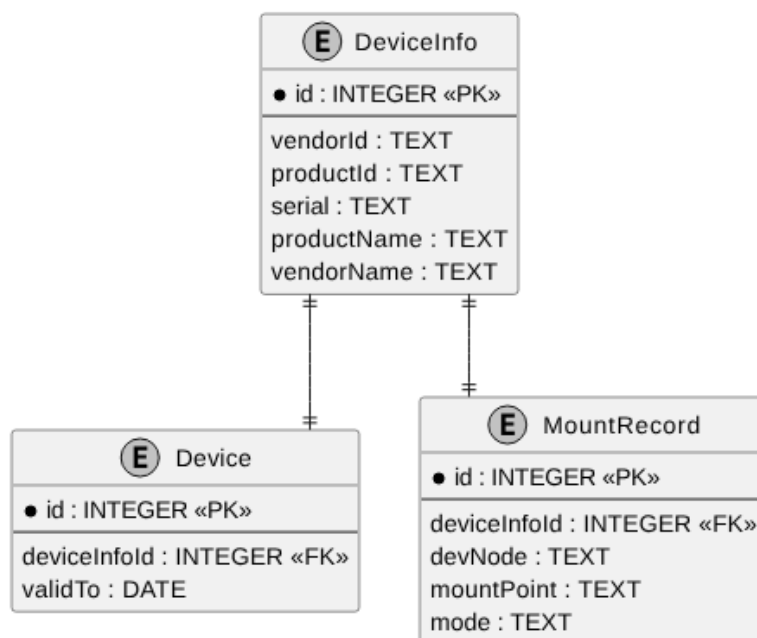


Рисунок 2.9 – Диаграмма базы данных

Между таблицами базы данных образуется связь один к одному.

2.10 Описание проектируемых ограничений целостности базы данных

На значения полей базы данных накладываются следующие ограничения целостности:

— **DeviceInfo:**

1. поля `vendorId` и `productId` должны содержать только цифры или строчные или заглавные латинские буквы, их длина должна равняться 4-ём символам [18], они не могут быть пустыми;
2. поле `serial` не может быть пустым;
3. комбинация полей `vendorId`, `productId`, `serial` должна быть уникальной.

— **MountRecord:**

1. поле `deviceId` является внешним ключом, ссылающимся на таблицу `DeviceInfo`, должно быть непустым и уникальным;
2. `devNode` должно быть непустым и уникальным;
3. `mountPoint` должно быть непустым и уникальным;
4. `mode` должно быть непустым и принимать только значения «RO» и «RW».

- **Device:** поле `deviceId` является внешним ключом, ссылающимся на таблицу `DeviceInfo`, должно быть непустым и уникальным, а `validTo` должно содержать дату в формате «ГГГГ-ММ-ДД».

3 Технологический раздел

В данном разделе приводится обоснование выбора языков программирования и среды разработки, средств тестирования кода. Описываются протоколы взаимодействия клиента и сервера, выбирается СУБД, приводятся классы эквивалентности, структура проекта. Также описывается взаимодействие пользователя с приложением и его развёртывание.

3.1 Выбор языков программирования и среды разработки

Для разработки серверной части программно-алгоритмического комплекса был выбран язык программирования C++ [19], что обусловлено следующими причинами:

1. встроенные возможности объектно-ориентированного программирования, позволяющие создавать легко расширяемые приложения;
2. совместимость с языком C, возможность прямого вызова функций мониторинга устройств;
3. поддержка многопоточности, необходимой для параллельной обработки событий;
4. стандартная библиотека предоставляет множество контейнеров, упрощающих разработку.

Для разработки клиентской части был выбран язык TypeScript [20] и фреймворк Vue.js [21].

Язык TypeScript обеспечивает статическую типизацию, что позволяет уменьшить количество ошибок на этапе разработки и повысить удобство сопровождения кода.

Выбор Vue.js обусловлен компонентным подходом к разработке пользовательского интерфейса, что упрощает повторное использование элементов приложения и повышает удобство сопровождения проекта. Дополнительно фреймворк предоставляет встроенные средства реактивного обновления интерфейса, позволяющие эффективно отображать изменения состояния приложения в режиме реального времени.

Для управления глобальным состоянием приложения была выбрана библиотека **Pinia** [22], являющаяся официальным инструментом управления состоянием для **Vue.js**. Использование **Pinia** позволяет централизованно хранить и обрабатывать данные приложения, упрощая взаимодействие между компонентами.

Для стилизации пользовательского интерфейса был выбран препроцессор **SASS** [23], расширяющий возможности стандартного **CSS (Cascading Style Sheets)**. Использование **SASS** позволяет применять переменные и повторно используемые стили, что упрощает разработку и сопровождение интерфейса приложения. Дополнительно препроцессор способствует повышению читаемости и структурированности стилей, а также уменьшает дублирование кода.

В качестве среды разработки был выбран **Visual Studio Code** [24] — кроссплатформенная среда, функциональность которой может быть увеличена с использованием дополнительных расширений.

3.2 Выбор средств тестирования кода

Для тестирования серверной части приложения был выбран фреймворк **GoogleTests** [25]. Данный продукт позволяет создавать **unit**-тесты, **e2e**-тесты и интеграционные тесты. Разработан специально для **C++**, и использует стандартный его синтаксис. Позволяет реализовывать **Mock**-объекты для изоляции зависимостей при тестировании отдельных компонентов системы [26]. Также фреймворк поддерживает параметризованные тесты, что позволяет проверять поведение методов на различных наборах входных данных без дублирования кода тестов. Для выполнения системно-зависимых тестов, таких как монтирование, тесты и приложение запускаются в контейнерах **Docker** [27].

Для тестирования клиентской части приложения была выбрана библиотека **Vitest** [28]. Данная библиотека предназначена для тестирования **JavaScript**- и **TypeScript**-приложений, а также является нативной для фреймворка **Vue.js**. **Vitest** поддерживает создание **unit**- и компонентных тестов, предоставляет встроенные средства мокирования функций и модулей. Дополнительно библиотека поддерживает параллельное выполнение тестов, что сокращает время тестирования клиентской части приложения.

3.3 Обеспечение связи между клиентом и сервером

Для связи клиента и сервера используется протокол HTTP (HyperText Transfer Protocol) [29] и механизм WebSocket [30].

Протокол HTTP является протоколом прикладного уровня модели OSI (Open Systems Interconnection) и предназначен для обмена данными между клиентом и сервером по схеме «запрос-ответ». Клиент формирует HTTP-запрос, сервер обрабатывает его и возвращает ответ, содержащий необходимые данные. Протокол широко применяется при передаче гипертекстовых документов и поддерживается большинством современных программных и аппаратных средств. В разрабатываемой программе используется, например, для получения списка подключённых устройств на клиенте.

Протокол WebSocket используется для организации постоянного двустороннего соединения между клиентом и сервером в режиме реального времени. В отличие от HTTP, WebSocket обеспечивает асинхронный обмен сообщениями и уменьшает накладные расходы на передачу данных за счёт отсутствия необходимости повторного создания соединения для каждого запроса. Установление WebSocket-соединения начинается с HTTP-запроса, после чего взаимодействие продолжается по отдельному протоколу WebSocket. В разрабатываемой программе используется для передачи на клиент информации о подключении и извлечении устройств.

3.4 Выбор программных средств для работы с устройствами и файловыми системами

Для взаимодействия с устройствами и файловыми системами в операционной системе Linux были выбраны библиотеки `libudev` [31], `libmount` [32] и `libblkid` [33]. Использование данных библиотек позволяет применять стандартные механизмы Linux для работы с USB-устройствами, операциями монтирования и определением типов файловых систем в пользовательском пространстве.

Библиотека `libudev` используется для получения информации об устройствах и отслеживания событий их подключения и отключения.

Библиотека `libmount` применяется для чтения файла `/proc/self/mountinfo` и определения режимов монтирования и точек

монтирования. Выбор `libmount` обусловлен тем, что библиотека предоставляет унифицированный интерфейс для работы с данными монтирования `Linux` и позволяет избежать ручного чтения системных файлов.

Библиотека `libblkid` применяется для получения типа файловой системы `USB`-накопителя перед выполнением операций монтирования.

3.5 Выбор встраиваемой системы управления реляционными базами данных

Для сравнения были выбраны следующие известные системы:

1. SQLite [34];
2. DuckDB [35];
3. H2 Database [36];
4. Apache Derby [37].

Результаты анализа существующих решений приведены в таблице 3.1.

Таблица 3.1 – Результат анализа существующих решений

Критерий	Номер решения			
	1	2	3	4
Поддержка <code>Linux</code>	да	да	да	да
Поддержка <code>C++</code>	да	да	нет	нет
Бесплатное распространение	да	да	да	да
Поддержка внешних ключей	да	да	да	да
Поддержка ограничений	да	да	да	да
Наличие обновлений и поддержки	да	да	да	нет
Отсутствие внешних зависимостей	да	да	нет	нет
Размер библиотеки(Мегабайты)	1.5	70	2.5	6.1

На основе анализа, для разрабатываемого комплекса была выбрана `SQLite`. Она полностью удовлетворяет всем критериям и имеет наименьший размер.

3.6 Классы эквивалентности

Для разработанного программно-алгоритмического комплекса были созданы основные классы эквивалентности.

3.6.1 Классы эквивалентности для получения списка подключённых устройств

Для всех классов маршрут `GET /api/v1/list`.

Класс наличия подключённых устройств:

Исходные данные: подключено 2 устройства с точками монтирования `/media/dlp/mnt1` и `/media/dlp/mnt2` соответственно, записи о монтировании есть в базе данных.

Ожидаемый результат: массив из 2 записей об устройствах с точками монтирования `/media/dlp/mnt1` и `/media/dlp/mnt2` соответственно.

Класс отсутствия подключённых устройств:

Исходные данные: подключённые устройства отсутствуют, база данных пуста.

Ожидаемый результат: пустой массив.

3.6.2 Классы эквивалентности для получения списка доверенных устройств

Для всех классов маршрут `GET /api/v1/whitelist`.

Класс наличия доверенных устройств:

Исходные данные: в базе данных есть 2 записи об устройствах с неистёкшими сроками доверия.

Ожидаемый результат: массив из 2 устройств.

Класс отсутствия доверенных устройств:

Исходные данные: доверенные устройства отсутствуют, база данных пуста.

Ожидаемый результат: пустой массив.

Класс наличия доверенных устройств и устройств с истёкшим сроком доверия:

Исходные данные: в базе данных есть 2 записи об устройствах с неистёкшими сроками доверия, и одна запись об устройстве с истёкшим сроком

доверия.

Ожидаемый результат: массив из 3 устройств.

3.6.3 Классы эквивалентности для изменения даты истечения доверия к устройству

Для всех классов маршрут PATCH /api/v1/whitelist/{id}.

Класс существующего идентификатора устройства и корректного формата даты:

Исходные данные: в базе данных есть запись об устройстве с идентификатором 1. Запрос содержит идентификатор 1, дата равна «2026-06-28».

Ожидаемый результат: в базе данных есть запись об устройстве с идентификатором 1 с изменённой на «2026-06-28» датой.

Класс несуществующего идентификатора устройства и корректного формата даты:

Исходные данные: в базе данных нет записи об устройстве с идентификатором 1. Запрос содержит идентификатор 1, дата равна «2026-05-28».

Ожидаемый результат: никаких изменений в базе данных.

Класс существующего идентификатора устройства и некорректного формата даты:

Исходные данные: в базе данных есть устройство с идентификатором 1. Запрос содержит идентификатор 1, дата равна «2026:05:28».

Ожидаемый результат: ошибка при записи в базу данных. Никаких изменений в базе данных.

3.6.4 Классы эквивалентности для удаления устройства из списка доверенных устройств

Для всех классов маршрут DELETE /api/v1/whitelist/{id}.

Класс существующего идентификатора несмонтированного устройства:

Исходные данные: в базе данных есть запись о доверенном устройстве с идентификатором 1, устройство не смонтировано. Запрос содержит идентификатор 1.

Ожидаемый результат: в базе данных отсутствует запись о доверенном устройстве с идентификатором 1.

Класс существующего идентификатора смонтированного устройства:

Исходные данные: в базе данных есть запись о доверенном устройстве с идентификатором 1, устройство смонтировано в режиме «Чтение и запись». Запрос содержит идентификатор 1.

Ожидаемый результат: в базе данных отсутствует запись о доверенном устройстве с идентификатором 1, устройство смонтировано в режиме «Только чтение».

Класс несуществующего идентификатора устройства:

Исходные данные: в базе данных отсутствует запись о доверенном устройстве с идентификатором 100. Запрос содержит идентификатор 100.

Ожидаемый результат: никаких изменений в базе данных.

3.6.5 Классы эквивалентности для добавления устройства в список доверенных устройств

Для всех классов маршрут POST /api/v1/whitelist/.

Класс добавления устройства, которого нет в списке доверенных:

Исходные данные: запрос содержит информацию об устройстве `info`, в базе данных нет устройства с таким `info`.

Ожидаемый результат: в базе данных есть запись о доверенном устройстве с таким `info`, устройство смонтировано в режиме «ReadWrite».

Класс добавления устройства, которое уже есть в списке доверенных:

Исходные данные: запрос содержит информацию об устройстве `info`, в базе данных есть устройство с таким `info`.

Ожидаемый результат: ошибка записи в базу данных.

3.6.6 Классы эквивалентности для метода построения имени папки монтирования

Класс наличия всех информационных полей устройства:

Входные данные: `vendorId = 1234`, `productId = 7856`, `serial = ACDC456IRHX`;

Ожидаемый результат: `/media/dlp/1234_7856_ACDC456IRHX`.

3.6.7 Классы эквивалентности для обработки события подключения устройства

Класс подключения недоверенного устройства:

Входные данные: событие подключения устройства `/dev/sda1`, данных о котором нет в таблице `Device`.

Ожидаемый результат: устройство смонтировано в режиме «Только для чтения».

Класс подключения доверенного устройства:

Входные данные: событие подключения устройства `/dev/sda1`, данные о котором есть в таблице `Device` и период доверия не истёк.

Ожидаемый результат: устройство смонтировано в режиме полного доступа.

Класс подключения устройства с истёкшим сроком доверия:

Входные данные: событие подключения устройства `/dev/sda1`, данные о котором есть в таблице `Device`, но период доверия истёк.

Ожидаемый результат: устройство смонтировано в режиме «Только для чтения».

3.6.8 Классы эквивалентности для получения записи о монтировании по специальному файлу устройства

Класс получения существующей записи о монтировании:

Входные данные: в хранилище содержится запись о монтировании для `/dev/sda1`.

Ожидаемый результат: запись о монтировании для `/dev/sda1`.

Класс получения записи о монтировании для несуществующего `/dev/sda1`:

Входные данные: в хранилище отсутствует запись о монтировании для `/dev/sda1`.

Ожидаемый результат: нулевой указатель.

3.7 Структура проекта серверной части комплекса

Исходный код приложения, а также вспомогательные и служебные элементы были логически разделены по функциональному признаку следующим

образом:

- `api-stubs/api/` — содержит в себе обработчики HTTP запросов;
- `external/` — директория для сторонних библиотек. Содержит `ixwebsocket` (библиотека для WebSocket-соединений) и `spdlog` (библиотека для журналирования);
- `sql/schema` — хранит скрипты для создания таблиц базы данных;
- `src/` — исходный код приложения:
 - `commands/` — содержит классы команд:
 - `Command` — базовый класс команды;
 - `GetWhiteListDeviceCommand` — команда получения списка доверенных устройств;
 - `AddDeviceToWhiteListCommand` — команда добавления устройства в список доверенных;
 - `DeleteDeviceFromWhiteListCommand` — команда удаления устройства из списка доверенных;
 - `PatchValidToDeviceCommand` — команда изменения даты истечения доверия устройства;
 - `GetCurrentConnectedDevicesCommand` — команда получения списка подключённых в данный момент устройств;
 - `CommandContext` — вспомогательный класс, содержащий необходимые сервисы для команд.
 - `core/` — содержит классы, реализующие основную работу по мониторингу устройств и обработки событий:
 - `DeviceControlService` — класс управляет обработкой события;
 - `EventLoop` — класс, отправляющий события из очереди на обработку;
 - `EventQueue` — класс, управляющий состоянием очереди событий;

- **IDeviceResolver** — интерфейс класса получения параметров устройств и монтирования;
 - **IMountSystem** — интерфейс класса операций монтирования/размонтирования;
 - **LinuxMountSystem** — класс операций монтирования/размонтирования;
 - **MountRecoveryService** — класс обновления записей о монтировании при запуске приложения;
 - **MountService** — класс, управляющий операциями монтирования;
 - **MountUtils** — класс-обёртка над операциями монтирования/размонтирования;
 - **UdevDeviceResolver** — класс получения параметров устройств и монтирования;
 - **UdevWatcher** — класс, отслеживающий события с устройствами.
- **essesnses/** — содержит классы-сущности и их классы породители:
 - **Device** — класс устройства;
 - **DeviceBuilder** — класс-создатель объекта устройства;
 - **DeviceEvent** — класс события подключения/отключения устройства;
 - **DeviceEventBuilder** — класс-создатель объекта события;
 - **DeviceInfo** — класс информации об устройстве;
 - **DeviceInfoBuilder** — класс-создатель объекта информации;
 - **MountRecord** — класс записи о монтировании;
 - **MountRecordBuilder** — класс-создатель объекта записи о монтировании.
- **facade/** — содержит класс **Facade**, унифицированный интерфейс доступа к подсистемам;
 - **managers/** — содержит классы **DeviceManager** и **MountRegistryManager**, предоставляющие доступ к классам репозитория;

- **repositories/** — содержит классы для работы с базой данных:
 - **DBConnection** — реализует подключение к базе данных, обёртку над функциями выполнения запросов;
 - **DeviceRepository** — реализует работу с таблицами **Device** и **DeviceInfo** базы данных;
 - **MountRepository** — реализует работу со списком записей о монтировании.
- **services/** — содержит вспомогательные классы:
 - **Config** — класс чтения конфигурации приложения;
 - **DBInitializer** — класс инициализации базы данных;
 - **DeviceEventNotifier** — класс отправки сообщений о событиях устройств на клиент;
 - **DevLogger** — класс логгера;
 - **BaseException** — базовый класс исключения;
 - **FileException** — класс исключения, связанного с файловой работой;
 - **HttpServerError** — класс исключения, связанного с HTTP сервером;
 - **SqlDataBaseError** — класс исключения, связанного с работой с базой данных;
 - **ResolveInfoError** — класс исключения, связанного с невозможностью извлечь данные об устройстве;
 - **UnknownFsError** — класс исключения, связанного с невозможностью установить тип файловой системы устройства;
 - **MountError** — класс исключения, связанного с операцией монтирования;
 - **UnmountError** — класс исключения, связанного с операцией размонтирования;
 - **HttpServer** — класс запуска HTTP сервера;
 - **IWebSocketServer** — интерфейс класса работы с **WebSocket**;
 - **MountPointBuilder** — класс создания пути монтирования;
 - **WebSocketServer** — класс работы с **WebSocket**.

- `main.cpp` — точка входа в программу.
- `tests/` — папка с тестами `GoogleTests`:
 - `e2e/` — End-To-End тесты;
 - `helpers/` — вспомогательные функции и классы, используемые в тестовом коде;
 - `integration/` — интеграционные тесты;
 - `mocks/` — мокированные объекты для тестов;
 - `unit/` — unit-тесты;
 - `CMakeLists.txt` — сценарий сборки тестового кода.
- `CMakeLists.txt` — сценарий сборки приложения;
- `docker-compose.yml` — конфигурационный файл `Docker`;
- `Dockerfile.app` — инструкции для создания образа контейнера приложения;
- `Dockerfile.e2e` — инструкции для создания образа контейнера для `e2e` тестов;
- `Dockerfile.tests` — инструкции для создания образа контейнера для `unit`- и интеграционных тестов.

3.8 Развёртывание приложения

Запуском серверной части управляет `systemd` [38]. Файл конфигурации `systemd` располагается по пути `/etc/systemd/system/usb-man.service`, его содержимое приведено в листинге 3.1.

Листинг 3.1 – Конфигурация systemd

```
[Unit]
Description=USBMan
[Service]
ExecStart=/usr/local/bin/usbman
Restart=always
[Install]
WantedBy=multi-user.target
```

Исполняемый файл сервера находится по пути `/usr/local/bin/usbman`, файл базы данных лежит в `/var/lib/usbman/app.db`, скрипты для создания таблиц находятся по пути `/usr/share/usbman/sql/`, файл конфигурации для приложения находится в `/etc/usbman/config.txt`, его содержимое приведено в листинге 3.2.

Листинг 3.2 – Конфигурация приложения

```
http.port=8080
websocket.port=9000
db.schema=/usr/share/usbman/sql/schema/001_device_info.sql;
           /usr/share/usbman/sql/schema/002_device.sql;
           /usr/share/usbman/sql/schema/003_mount_record.sql
db.path=/var/lib/usbman/app.db
```

Запуск клиентского приложения выполняется с помощью `systemd` и сервера `Nginx` [39]. Конфигурация `Nginx` располагается в файле `/etc/nginx/sites-available/usb-ui`, её содержимое приведено в листингах 3.3- 3.4.

Листинг 3.3 – Конфигурация Nginx

```
server {
    listen 5173;
    root /var/www/usb-ui;
    index index.html;

    location / {
        try_files $uri /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8080;
    }
}
```

Листинг 3.4 – Продолжение листинга 3.3

```
location /ws {
    proxy_pass http://127.0.0.1:9000;

    proxy_http_version 1.1;

    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
}
```

Файлы, отдаваемые Nginx лежат в папке `/var/www/usb-ui`. В ней находятся собранные `*.html`, `*.js`, `*.css` и графические файлы иконок.

3.9 Взаимодействие пользователя с приложением

Пользователь может взаимодействовать с программным комплексом через Web-интерфейс, доступный по адресу `http://localhost:5173/`.

3.9.1 Экран подключённых устройств

По адресу `http://localhost:5173/list/` пользователю доступен список подключённых в данный момент устройств. Над списком есть 2 кнопки для перехода по экранам приложения. Слева находится кнопка с надписью «Подключённые устройства» (переход на описываемый экран), а справа от неё кнопка с надписью «Доверенные устройства» (переход в список доверенных устройств). Для каждого устройства отображаются следующие данные:

1. путь монтирования;
2. режим монтирования;
3. название производителя;
4. название устройства;
5. серийный номер;
6. идентификатор производителя;
7. идентификатор устройства.

Также справа от каждого устройства отображается иконка добавления в список доверенных устройств с изображением знака +, при наведении указателя мыши на которую всплывает подсказка с текстом «Добавить». В случае, если устройство уже доверено, справа отображается иконка с изображением знака ✓, при наведении курсора на которую всплывает подсказка с текстом «Уже добавлено». При наведении указателя мыши на любую из кнопок курсор меняется на символ . Макет описанного экрана представлен на рисунке 3.1.

Подключённые устройства							
Путь монтирования	Режим	Производитель	Имя устройства	Серийный номер	Идентификатор производителя	Идентификатор продукта	
/media/usb1	Только чтение	Kingston	DataTraveler 3.0	DTX-001245	0951	1666	+
/media/usb2	Полный доступ	SanDisk	Ultra USB	SDK-882341	0781	5581	✓
/media/usb3	Только чтение	Transcend	JetFlash	TR-771245	8564	1000	+
/media/usb4	Полный доступ	Samsung	BAR Plus	SM-552341	04E8	61B6	✓
/media/usb5	Только чтение	ADATA	UV150	AD-982341	125F	C93A	+
/media/usb6	Полный доступ	Apacer	АН333	AP-112233	1005	B113	✓
/media/usb7	Только чтение	Silicon Power	Blaze B05	SP-554433	13FE	5500	+
/media/usb8	Полный доступ	Verbatim	Store n Go	VB-765421	18A5	0243	✓

Рисунок 3.1 – Экран подключённых в данный момент устройств

3.9.2 Экран доверенных устройств

По адресу <http://localhost:5173/whitelist/> пользователю доступен список доверенных устройств. Над списком аналогично находятся кнопки навигации. Для каждого устройства отображаются следующие параметры:

1. название производителя;
2. название устройства;
3. серийный номер;
4. идентификатор производителя;
5. идентификатор устройства;

6. дата истечения доверия.

В случае если дата истечения доверия не задана (устройство доверено бессрочно), то в поле «Доверено до» отображается шаблон даты **дд.мм.гггг**.

Также справа от каждого устройства отображается иконка удаления из списка доверенных устройств с изображением символа , при наведении указателя мыши на которую всплывает подсказка с текстом «Удалить».

Макет описанного экрана представлен на рисунке 3.2.

Подключенные устройства

Доверенные устройства

Доверенные устройства

Производитель	Имя устройства	Серийный номер	Идентификатор производителя	Идентификатор продукта	Доверено до	
Kingston	DataTraveler 3.0	DTX-001245	0951	1666	31.12.2026	 
SanDisk	Ultra USB	SDK-882341	0781	5581	дд.мм.гггг	 
Samsung	BAR Plus	SM-552341	04E8	61B6	10.05.2027	 
ADATA	UV150	AD-982341	125F	C93A	дд.мм.гггг	 
Corsair	Flash Voyager	CR-998877	1B1C	1A90	15.08.2026	 
PNY	Turbo Attaché	PN-334455	154B	6545	01.01.2028	 
Transcend	JetFlash	TR-771245	8564	1000	дд.мм.гггг	 
Verbatim	Store n Go	VB-765421	18A5	0243	20.11.2027	 
Silicon Power	Blaze B05	SP-554433	13FE	5500	дд.мм.гггг	 
Apacer	AH333	AP-112233	1005	B113	30.09.2026	 

Рисунок 3.2 – Экран доверенных устройств

При нажатии на поле «Доверено до» появляется всплывающее окно календаря, в котором пользователь может указать дату истечения доверия, которая не может быть раньше текущей. В случае, если необходимо сделать доверие бессрочным, можно нажать на кнопку календаря с надписью «Удалить». Макет выбора даты представлен на рисунке 3.3.

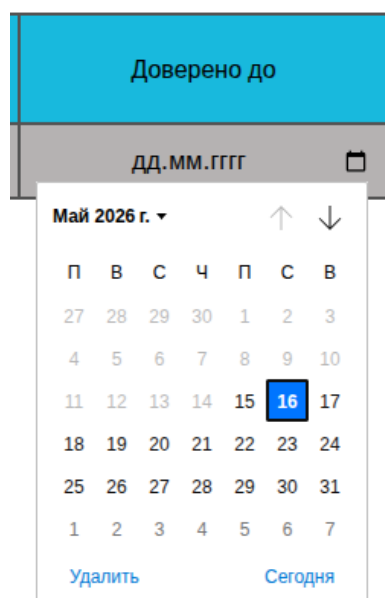


Рисунок 3.3 – Экран выбора даты истечения доверия в календаре

3.10 Результаты тестирования

Общее покрытие строк кода клиентской части составляет 92%. Детальный отчёт о покрытии представлен на рисунке 3.4.

All files

91.66% Statements 119/128 **76.31%** Branches 29/38 **88.37%** Functions 38/43 **91.96%** Lines 183/112

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File ▲		Statements ⇅		Branches ⇅		Functions ⇅		Lines ⇅	
components		91.42%	32/35	83.33%	20/24	84.61%	11/13	93.54%	29/31
services		100%	5/5	100%	2/2	100%	1/1	100%	5/5
stores		84.44%	38/45	58.33%	7/12	80%	12/15	83.72%	36/43
views		100%	35/35	100%	0/0	100%	14/14	100%	33/33

Рисунок 3.4 – Отчёт о покрытии кода клиента

Общее покрытие строк кода серверной части составляет 94%. Детальный отчёт о покрытии представлен на рисунке 3.5.

LCOV - code coverage report

Current view: [top level](#)

Test: [coverage.filtered.info](#)

Date: [2026-05-30 21:51:49](#)

	Hit	Total	Coverage
Lines:	542	576	94.1 %

Functions:	111	115	96.5 %
------------	-----	-----	--------

Directory	Line Coverage			Functions	
commands		100.0 %	14 / 14	80.0 %	4 / 5
core		91.2 %	125 / 137	93.8 %	15 / 16
essenses		94.5 %	52 / 55	95.2 %	20 / 21
facade		100.0 %	12 / 12	100.0 %	4 / 4
managers		100.0 %	13 / 13	100.0 %	12 / 12
repositories		92.6 %	212 / 229	97.0 %	32 / 33
services		98.3 %	114 / 116	100.0 %	24 / 24

Generated by: [LCOV version 1.14](#)

Рисунок 3.5 – Отчёт о покрытии кода сервера

4 Исследовательский раздел

В данном разделе проводится исследование временных характеристик разработанного программного обеспечения.

4.1 Постановка исследования

Предметом исследования является время выполнения алгоритма обработки специальных файлов устройств и записей о точках монтирования при инициализации приложения при различных начальных условиях. В первой части исследования устанавливается взаимосвязь между временем выполнения и количеством исходных актуальных записей о монтировании. Вторая часть исследования направлена на влияние на время выполнения количества записей о монтировании, которые необходимо обновить. Третья часть исследования устанавливает взаимосвязь между временем выполнения и количеством необходимых вызовов перемонтирования.

4.2 Технические характеристики

Исследование проводилось на виртуальной машине с операционной системой **Linux Ubuntu Server**. Для создания виртуальной машины использовался программный механизм KVM (Kernel-based Virtual Machine) [40].

Конфигурация виртуальной машины:

- количество процессоров — 4;
- объём оперативной памяти — 6 гигабайт;
- объём виртуального диска — 20 гигабайт.

Хостовая машина работает на процессоре 11th Gen Intel(R) Core(TM) i7-11370H @ 3.30 ГГц [41].

4.3 Проведение исследования и результаты

Для проведения исследования с помощью модуля ядра `dummy_hcd` [42] было создано виртуальное устройство USB с восьмью LUN (Logical Unit Number).

4.3.1 План первой части исследования

Количество исходных записей о монтировании N изменяется от 0 до 8. Для каждого состояния проводится 50 замеров времени и их дальнейшее усреднение. Перед каждым замером очищается хранилище записей о монтировании, все устройства размонтируются, N устройств из 8 монтируется, записи о них сохраняются. Количество виртуальных устройств неизменно.

4.3.2 Результаты первой части исследования

В результате были получены данные, отражённые в таблице 4.1.

Таблица 4.1 – Результаты первой части исследования

Количество записей о монтировании	Время выполнения
0	62.3551
1	61.2938
2	59.9159
3	55.7733
4	53.0081
5	49.1145
6	46.3936
7	42.8975
8	37.9524

По табличным значениям был построен график зависимости времени выполнения алгоритма обработки специальных файлов устройств и записей о точках монтирования от количества исходных записей о точках монтирования, представленный на рисунке 4.1.

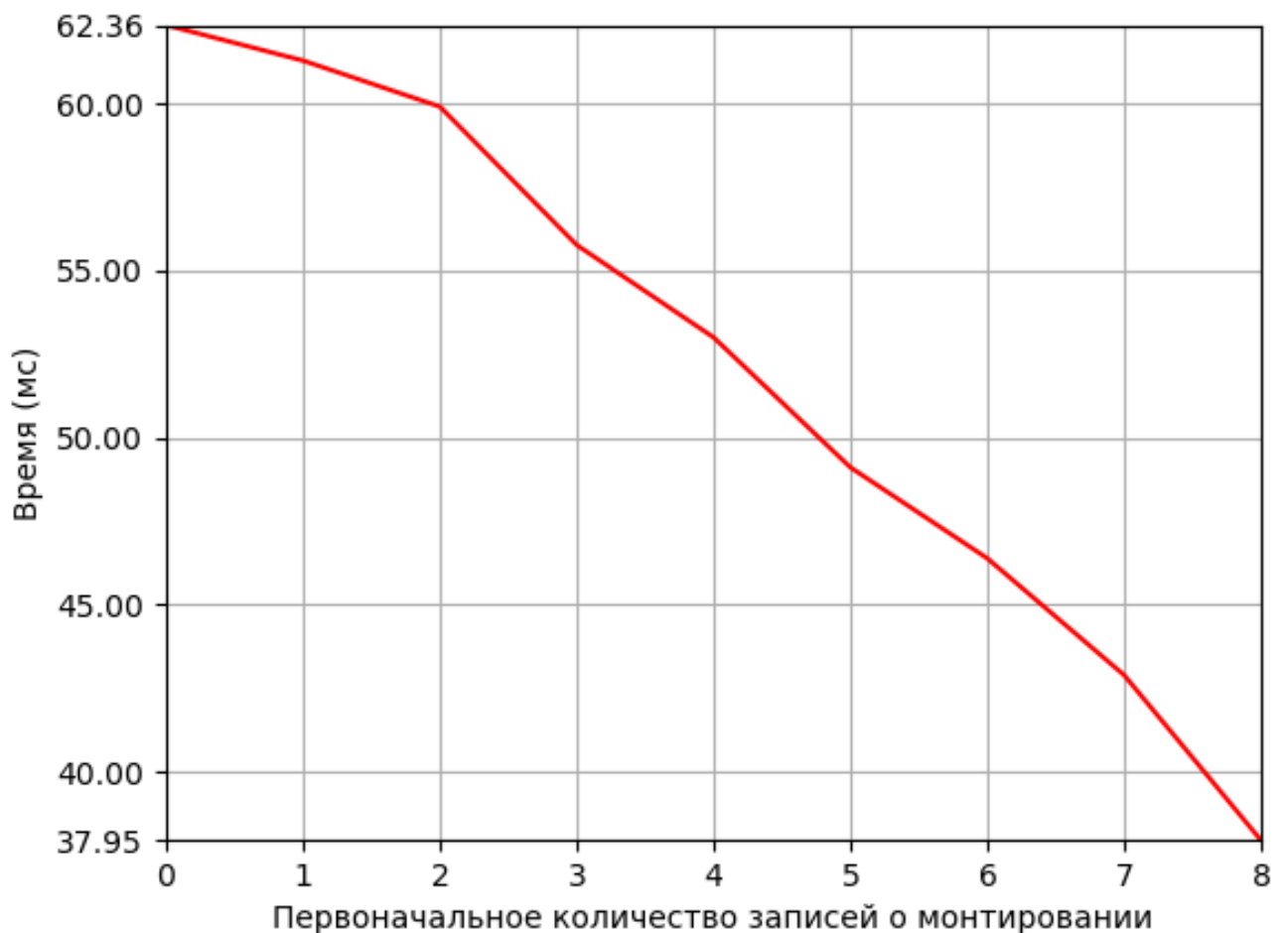


Рисунок 4.1 – График по результатам первой части исследования

Время выполнения алгоритма практически линейно зависит от количества записей. При наличии всех записей о монтировании время выполнения уменьшается примерно в два раза относительно состояния отсутствия записей, так как отсутствует необходимость в монтировании и записи.

4.3.3 План второй части исследования

Количество исходных записей о монтировании и количество виртуальных устройств неизменно и равно 8. Для каждого состояния проводится 50 замеров времени и их дальнейшее усреднение. Перед каждым замером очищается хранилище записей о монтировании, все устройства перемонтируются, часть записей сохраняется в достоверном виде, а N записей записи сохраняются в изменённом формате — меняется путь монтирования, где N изменяется от 0 до 8.

4.3.4 Результаты второй части исследования

В результате были получены данные, отражённые в таблице 4.2.

Таблица 4.2 – Результаты второй части исследования

Количество недостоверных записей	Время выполнения
0	35.8714
1	36.6813
2	36.9145
3	37.2408
4	37.3638
5	38.7439
6	39.9057
7	41.629
8	43.7524

По табличным значениям был построен график зависимости времени выполнения алгоритма обработки узлов устройств и записей о точках монтирования от количества недостоверных записей о монтировании, представленный на рисунке 4.2.

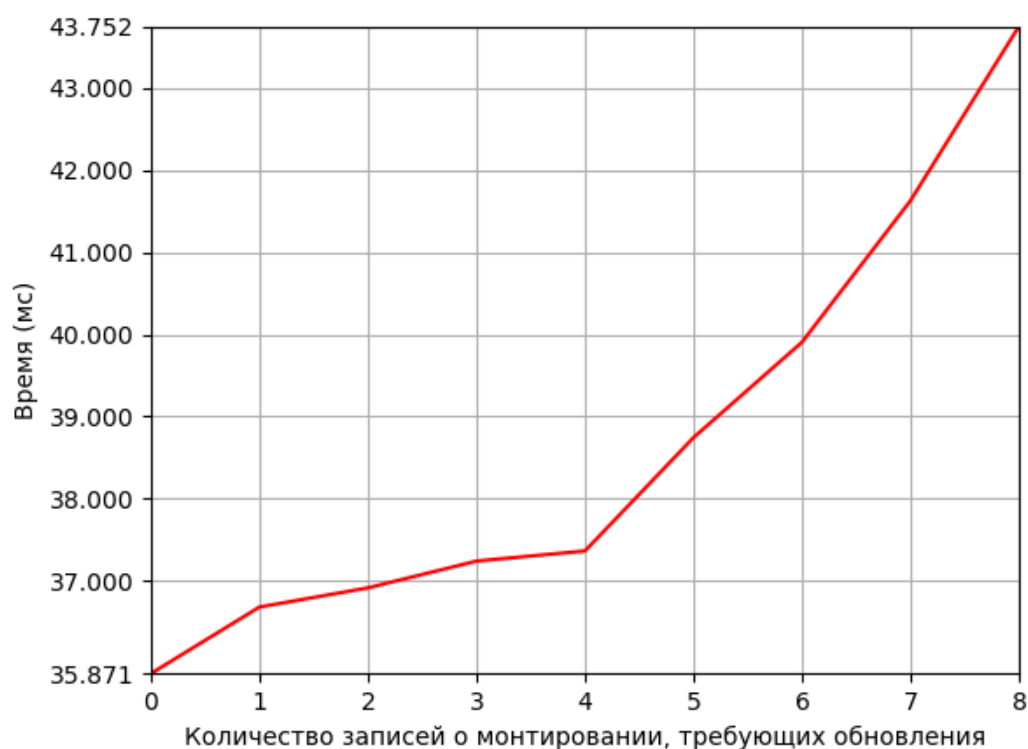


Рисунок 4.2 – График по результатам второй части исследования

При обнаружении несоответствия фактического пути монтирования и документированного пути происходит обновление записи о монтировании, что отражается на росте времени выполнения алгоритма.

4.3.5 План третьей части исследования

Количество исходных записей о монтировании и количество виртуальных устройств неизменно и равно 8. Для каждого состояния проводится 50 замеров времени и их дальнейшее усреднение. Перед каждым замером очищается хранилище записей о монтировании, все устройства перемонтируются. Часть устройств монтируется в режиме `ReadOnly`, а N в режиме `ReadWrite`, где N изменяется от 0 до 8. При этом в список доверенных устройств ничего не записывается, то есть алгоритм будет выявлять несоответствие между `ReadWrite` режимом монтирования устройства и его отсутствия в списке доверенных, и перемонтировать в режиме `ReadOnly`.

4.3.6 Результаты третьей части исследования

В результате были получены данные, отражённые в таблице 4.3.

Таблица 4.3 – Результаты третьей части исследования

Количество устройств, смонтированных в режиме <code>ReadWrite</code>	Время выполнения
0	38.9785
1	41.3014
2	42.3494
3	43.8505
4	44.3037
5	44.6508
6	46.7866
7	48.4142
8	48.5326

По табличным значениям был построен график зависимости времени выполнения алгоритма обработки узлов устройств и записей о точках монтирования от количества недостоверных записей о монтировании, представленный

на рисунке 4.3.

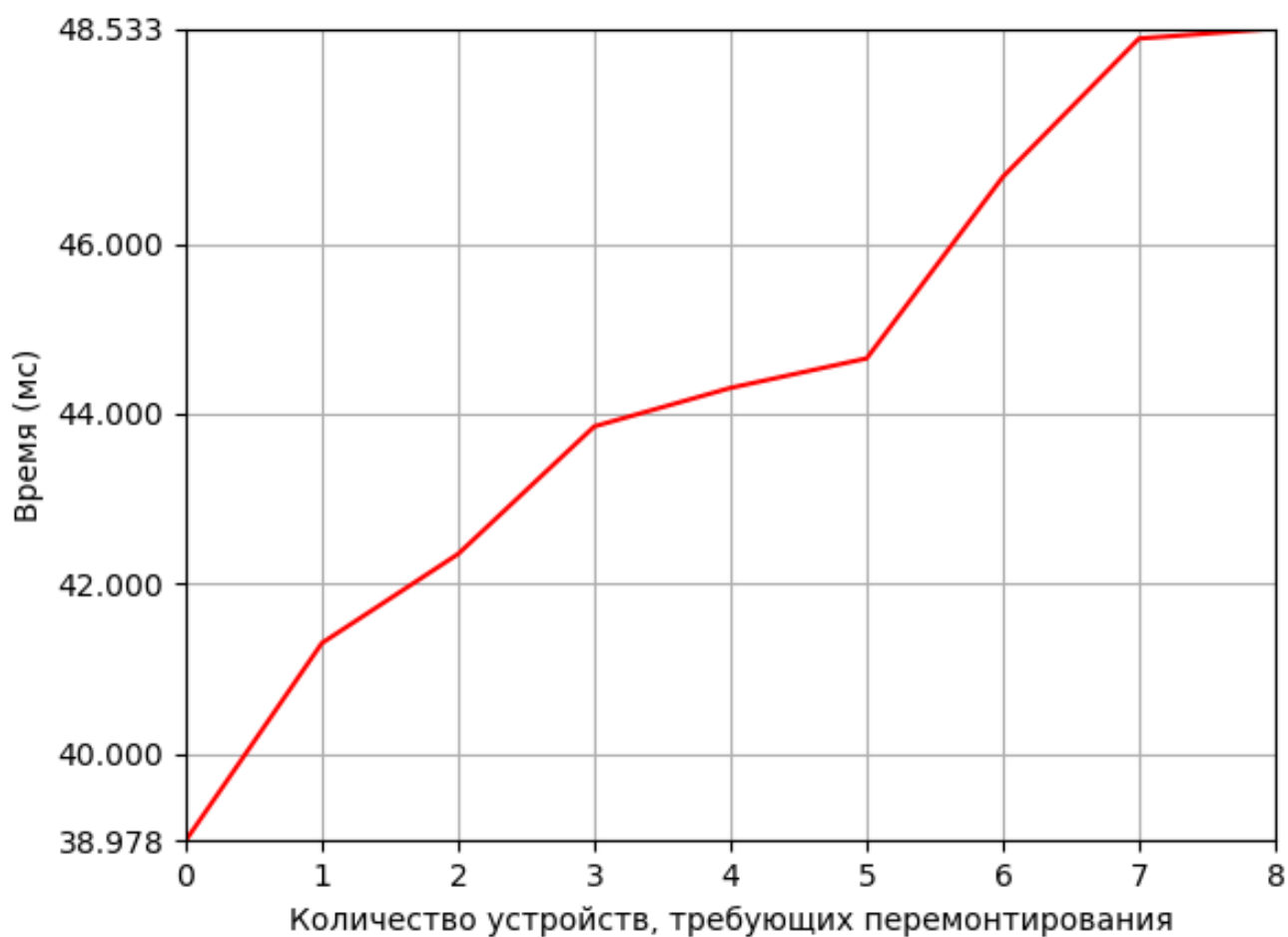


Рисунок 4.3 – График по результатам третьей части исследования

Количество несоответствий в режиме монтирования влияет на количество операций перемонтирования, что влечёт за собой увеличение времени выполнения алгоритма.

ЗАКЛЮЧЕНИЕ

Таким образом, цель дипломной работы была достигнута — разработан и реализован программно-алгоритмический комплекс для предотвращения несанкционированного копирования информации посредством управления доступом к USB-носителям.

Выполнены все поставленные задачи:

- проанализированы существующие решения в сфере управления доступом ко внешним запоминающим устройствам;
- формализованы требования к разрабатываемому комплексу с использованием диаграммы прецедентов;
- разработана архитектура программно-алгоритмического комплекса;
- описаны компоненты и их взаимодействие;
- описаны ключевые структуры данных;
- обоснован выбор средств программной реализации;
- разработано программное обеспечение комплекса;
- проведено тестирования комплекса;
- описано взаимодействие пользователя с программным обеспечением;
- исследованы характеристики разработанного программного обеспечения.

Результаты исследования показали, что:

1. количество актуальных записей о монтировании линейно влияет на время работы алгоритма актуализации записей;
2. необходимость обновления информации в записи о монтировании увеличивает время инициализации приложения;
3. необходимость перемонтирования устройств при инициализации приложения также увеличивает время инициализации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Баев А. В., Гаценко О. Ю., Самонов А. В. Программный комплекс управления доступом USB-устройств к автоматизированным рабочим местам // Вопросы кибербезопасности. — 2020. — №1 (35).
2. InfoWatch — Аналитика информационной безопасности [Электронный ресурс]. — Режим доступа: <https://www.infowatch.ru/analytics> (дата обращения: 11.11.2025).
3. USBGuard [Электронный ресурс]. — Режим доступа: <https://usbguard.github.io/> (дата обращения: 12.01.2026).
4. Making USB Great Again with USBFILTER — a USB layer firewall in the Linux kernel [Электронный ресурс]. — Режим доступа: <https://davejingtian.org/2016/08/04/making-usb-great-again-with-usbfilter-a-usb-layer-firewall-in-the-linux-kernel/> (дата обращения: 12.01.2026).
5. USBWall Source Code [Электронный ресурс]. — Режим доступа: <https://github.com/usbwall/usbwall?ysclid=mndivt3urh835120502> (дата обращения: 12.01.2026).
6. USBShield Source Code [Электронный ресурс]. — Режим доступа: <https://github.com/USBShield/USBShield?ysclid=mn diy aq4ah326619085> (дата обращения: 12.01.2026).
7. Sordum — Ratool v1.4 (Removable Access tool) [Электронный ресурс]. — Режим доступа: <https://www.sordum.org/8104/ratool-v1-4-removable-access-tool/> (дата обращения: 12.01.2026).
8. Gilisoft USB Lock — Control USB ports, removable devices, and trusted-device access to reduce data leakage on Windows [Электронный ресурс]. — Режим доступа: <https://www.gilisoft.com/product-usb-lock.htm> (дата обращения: 12.01.2026).
9. NewSoftwares — USBBlock. Restrict Access of Portable Drives [Электронный ресурс]. — Режим доступа: <https://www.newsoftwares.net/usb-block/> (дата обращения: 12.01.2026).
10. Робачевский А.М. Операционная система UNIX. — Санкт-Петербург : БХВ-Петербург, 2001. — С. 528.

11. Таненбаум Э., Бос Х. Современные операционные системы. — Санкт-Петербург : 2024. — С. 1120.
12. Russell R., Quinlan D., Yeoh C. Filesystem hierarchy standard //V2. — 2004. — Т. 3. — С. 29.
13. Руководство программиста Linux — MOUNT(2) [Электронный ресурс]. — Режим доступа: <https://manpages.debian.org/bullseye-backports/manpages-ru-dev/mount.2.ru.html> (дата обращения: 01.03.2026).
14. Руководство программиста Linux — UMount(2) [Электронный ресурс]. — Режим доступа: <https://manpages.debian.org/bullseye-backports/manpages-ru-dev/umount.2.ru.html> (дата обращения: 01.03.2026).
15. Модели баз данных: учебное пособие / О.Е. Аврунев, В.М. Стасышин. — Новосибирск : Изд-во НГТУ, 2018. — С. 124.
16. Благовещенский В. Г. Создание базы данных для разработки облачной платформы хранения и редактирования 3D моделей конфет / Благовещенский В. Г., Краснов А. Е., Благовещенский И. Г. // Интеллектуальные автоматизированные управляющие системы в биотехнологических процессах : сборник докладов всероссийской научно-практической конференции, Москва, 29 марта 2023 года. — Москва : РОССИЙСКИЙ БИОТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ; ЗАО «Университетская книга», 2023. — С. 85-96.
17. Документация OpenAPI Initiative — The OpenAPI Initiative [Электронный ресурс]. — Режим доступа: <https://www.openapis.org/> (дата обращения: 17.03.2026).
18. Документация USB — USB 2.0 Specification [Электронный ресурс]. — Режим доступа: <https://www.usb.org/document-library/usb-20-specification> (дата обращения: 19.03.2026).
19. Документация C++ — Standard C++ [Электронный ресурс]. — Режим доступа: <https://isocpp.org/> (дата обращения: 01.04.2026).
20. Документация TypeScript — JavaScript With Syntax For Types [Электронный ресурс]. — Режим доступа: <https://www.typescriptlang.org/> (дата обращения: 01.04.2026).

21. Документация Vue.js — The Progressive JavaScript Framework | Vue.js [Электронный ресурс]. — Режим доступа: <https://vuejs.org/> (дата обращения: 01.04.2026).
22. Документация Pinia — The intuitive store for Vue.js [Электронный ресурс]. — Режим доступа: <https://pinia.vuejs.org/> (дата обращения: 01.04.2026).
23. Документация Sass — Syntactically Awesome Style Sheets [Электронный ресурс]. — Режим доступа: <https://sass-lang.com/> (дата обращения: 01.04.2026).
24. Документация Visual Studio Code — The open source AI code editor [Электронный ресурс]. — Режим доступа: <https://code.visualstudio.com/> (дата обращения: 02.04.2026).
25. Документация GoogleTest User's Guide [Электронный ресурс]. — Режим доступа: <https://google.github.io/googletest/> (дата обращения: 22.04.2026).
26. Тюменцев Е. А. Автоматизированное тестирование сложности алгоритмов с помощью Mock-объектов / Тюменцев Е. А. // Математические структуры и моделирование. — 2013. — № 1(27). — С. 82-88.
27. Документация Docker — Accelerated Container Application Development [Электронный ресурс]. — Режим доступа: <https://www.docker.com/> (дата обращения: 22.04.2026).
28. Документация Vitest — Next Generation testing framework [Электронный ресурс]. — Режим доступа: <https://vitest.dev/> (дата обращения: 22.04.2026).
29. Борисов П. Е. HTTP-протокол прикладного уровня (обзор) // Научный аспект. — 2024. — Т. 22. — №. 5. — С. 2937-2943.
30. Хабаров С. П. Взаимодействия узлов сети по протоколу WebSocket / Хабаров С. П. // Информационные системы и технологии: теория и практика : Сборник научных трудов научно-технической конференции института леса и природопользования, Санкт-Петербург, 01 февраля 2017 года. Том Выпуск 9. — Санкт-Петербург : Санкт-Петербургский

- государственный лесотехнический университет им. С.М. Кирова, 2017. — С. 104-115.
31. Документация libudev(3) — Linux manual page [Электронный ресурс]. — Режим доступа: <https://man7.org/linux/man-pages/man3/libudev.3.html> (дата обращения: 11.04.2026).
 32. Документация ядра Linux — libmount Reference Manual [Электронный ресурс]. — Режим доступа: <https://www.kernel.org/pub/linux/utils/util-linux/v2.36/libmount-docs/index.html> (дата обращения: 12.04.2026).
 33. Документация libblkid(3) — Linux manual page [Электронный ресурс]. — Режим доступа: <https://man7.org/linux/man-pages/man3/libblkid.3.html> (дата обращения: 15.04.2026).
 34. Документация SQLite Home Page [Электронный ресурс]. — Режим доступа: <https://www.sqlite.org/> (дата обращения: 17.04.2026).
 35. Документация DuckDB — An in-process SQL OLAP database management system [Электронный ресурс]. — Режим доступа: <https://duckdb.org/> (дата обращения: 17.04.2026).
 36. Документация H2 Database Engine — Welcome to H2, the Java SQL database [Электронный ресурс]. — Режим доступа: <https://h2database.com/html/main.html> (дата обращения: 17.04.2026).
 37. Документация Apache Derby [Электронный ресурс]. — Режим доступа: <https://db.apache.org/derby/> (дата обращения: 17.04.2026).
 38. Документация Systemd — System and Service Manager [Электронный ресурс]. — Режим доступа: <https://systemd.io/> (дата обращения: 29.04.2026).
 39. Документация nginx [Электронный ресурс]. — Режим доступа: <https://nginx.org/ru/> (дата обращения: 29.04.2026).
 40. Документация KVM — Main Page [Электронный ресурс]. — Режим доступа: https://linux-kvm.org/page/Main_Page (дата обращения: 01.05.2026).

41. Документация Intel Core i7-11370H Specs — TechPowerUp CPU Database [Электронный ресурс]. — Режим доступа: <https://www.techpowerup.com/cpu-specs/core-i7-11370h.c2833> (дата обращения: 01.05.2026).
42. Исходный код ядра Linux (v7.0.8) — Bootlin Elixir Cross Referencer [Электронный ресурс]. — Режим доступа: https://elixir.bootlin.com/linux/v6.7-rc3/source/drivers/usb/gadget/udc/dummy_hcd.c#L82 (дата обращения: 02.05.2026).

ПРИЛОЖЕНИЕ А

Диск с рабочими материалами со следующей структурой каталогов:

- `report` — каталог с РПЗ ВКР;
- `app/` — каталог с исходными кодами сервера:
 - `api-stubs` — обработчики HTTP запросов;
 - `external` — сторонние библиотеки;
 - `sql/schema` — скрипты для создания таблиц базы данных;
 - `src` — исходный код приложения;
 - `tests` — тесты;
 - `CMakeLists.txt` — сценарий сборки приложения;
 - `docker-compose.yml` — конфигурационный файл Docker;
 - `Dockerfile.app` — инструкции для создания образа контейнера приложения;
 - `Dockerfile.e2e` — инструкции для создания образа контейнера для e2e тестов;
 - `Dockerfile.tests` — инструкции для создания образа контейнера для unit- и интеграционных тестов.
- `front/` — каталог с исходными кодами клиента:
 - `src` — исходный код клиента;
 - `index.html` — основная вёрстка приложения;
 - `package.json` — файл зависимостей;
 - `tsconfig.json` — конфигурация TypeScript;
 - `vite.config.ts` — конфигурация сборщика Vite;
 - `vitest.config.ts` — конфигурация тестов.

ПРИЛОЖЕНИЕ Б

Презентация к выпускной квалификационной работе на тему «Программно-алгоритмический комплекс управления доступом ко внешним запоминающим устройствам» состоит из 18 слайдов.